

# Three ways to search binary embeddings

From stored data to query latency,  
memory usage, and recall

- A** Cluster, then score every selected document.
- B** Cluster, then traverse an inverted bitplane index.
- C** Cluster, then enumerate keys from the query's ideal sign pattern.

This report develops the three methods from the same score and the same retrieval task. It explains what is built before a query, what is touched during a query, and where each method can lose recall. Every general cost formula uses named parameters. Small numerical examples make the formulas concrete; measured results are identified separately.

**Reading order.** Start with the four-bit example and shared pipeline. Each method then has its own data layout, pseudocode, operation count, latency model, and memory calculation. The statistical section explains what can and cannot be inferred about recall. The final section collects the formulas for comparison under a RAM limit.

## Contents

|    |                                                             |    |
|----|-------------------------------------------------------------|----|
| 1  | Start with the question a search must answer                | 2  |
| 2  | What is built once, and what every query pays               | 6  |
| 3  | Approach A: cluster, then full scan                         | 10 |
| 4  | Approach B: cluster, then inverted bitplanes and branching  | 14 |
| 5  | Approach C: cluster, then walk backwards from the ideal key | 18 |
| 6  | Deriving recall without assuming the answer                 | 26 |
| 7  | What the existing measurements support                      | 39 |
| 8  | From the formulas to an implemented comparison              | 45 |
| 9  | A distribution where the important coordinates are known    | 50 |
| 10 | Check the complete cost, not just the counters              | 55 |
| 11 | Bring the formulas together                                 | 59 |
| A  | Computing the expected top-K count at each distance         | 67 |
| B  | A finite-sample view of prefix recall                       | 67 |
| C  | Checklist for an honest numerical implementation            | 68 |
| D  | Floating-point arithmetic and exactness                     | 69 |
| E  | Optional expansion of the embedding cost                    | 69 |
| F  | Reproduction and parameter ledger                           | 70 |

# 1 Start with the question a search must answer

Suppose a database contains  $N$  documents. A query arrives, and the system must return  $K$  documents. We want the smallest query latency that fits the available RAM and reaches a specified recall target. Before comparing algorithms, we must agree on what “the correct  $K$  documents” means.

This report first uses one precise target: the top  $K$  documents under the same **float-query, binary-document inner product**. This isolates the search index. A later reranking step may use longer float embeddings, but that is a different score and a different reference ranking. Exact binary search need not recover the exact float ranking, and neither ranking is automatically the same as human relevance.

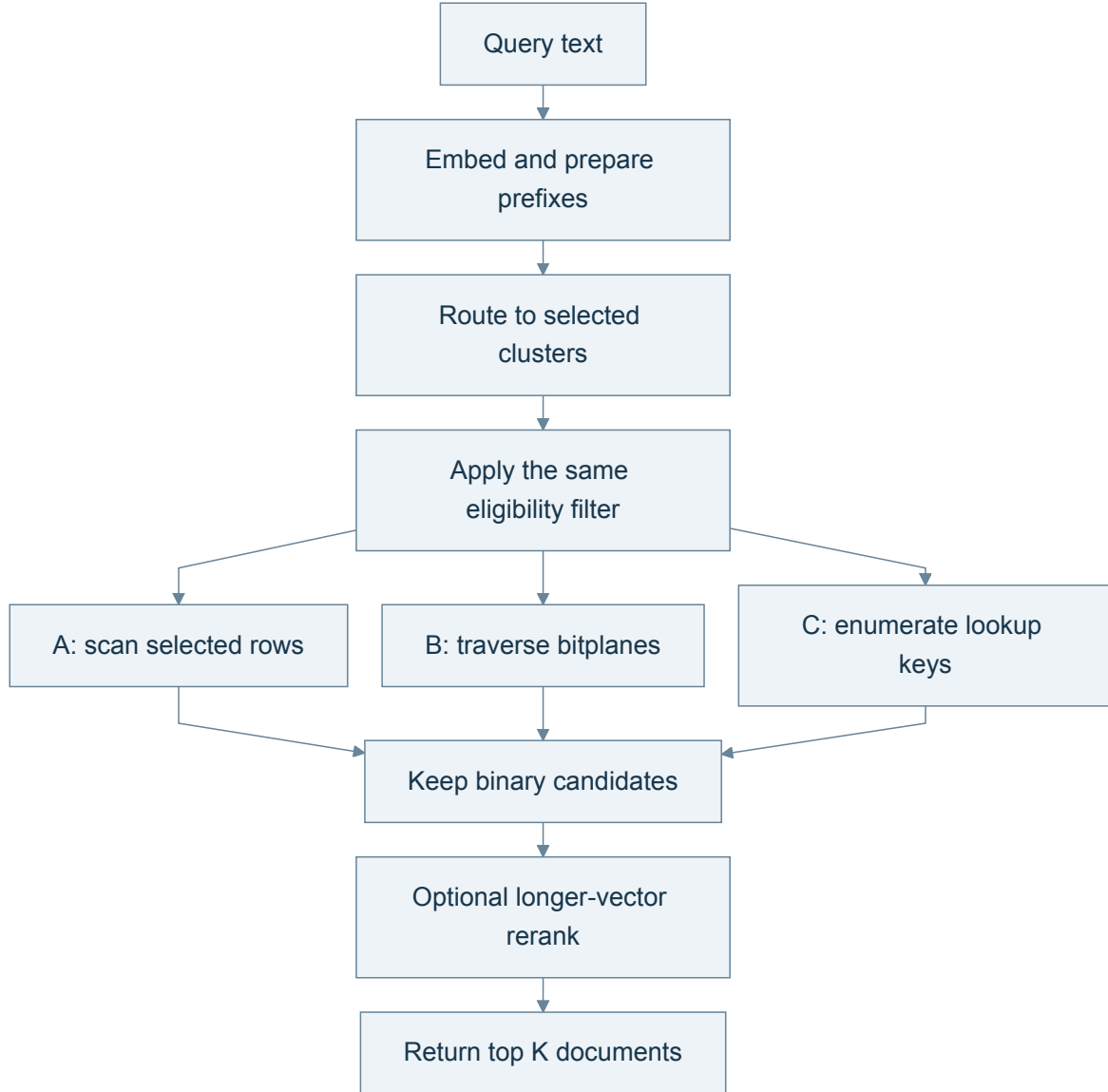
## 1.1 Three costs that must not be confused

1. **Index storage:** data built in advance and kept between queries. Examples are packed document codes, cluster offsets, centroids, bitplanes, and key directories.
2. **Query RAM:** temporary live data needed by one query. Examples are a query vector, score tables, a top- $R$  result heap, pending branch masks, and key-enumeration states.
3. **Query work:** operations and data accesses over time. Reading the same array a thousand times does not require a thousand stored copies, but it can cost a thousand reads.

The number of scored documents alone does not measure work. A method can score fewer documents and still read more memory or manage a much larger queue.

## 1.2 The shared pipeline

All three primary methods begin with clusters. The no-clustering experiment is the special case  $C = P = 1$ , not a different definition of the task.



**Figure 1:** The common query pipeline. Only the candidate-search stage changes between A, B and C.

An embedding model converts query text to a float vector. A short representation routes the query to selected clusters. Optional filters select eligible documents. The index produces up to  $R$  binary-score candidates, with  $R \geq K$ . An optional final stage fetches longer vectors and reranks those candidates, then returns  $K$  results.

If the experiment only measures the index, cache the query embedding and omit the optional reranker. If the experiment claims end-to-end query latency, include them. We will keep both versions visible in the formulas.

### 1.3 A four-bit example we can calculate by hand

Let

$$q = (0.7, -0.4, 0.2, -0.1), \quad t = \text{sign}(q) = (+1, -1, +1, -1).$$

Write a stored  $+1$  as bit 1 and  $-1$  as bit 0. The ideal bit pattern is therefore 1010. Define  $a_i = |q_i|$  and  $A = \sum_i a_i = 1.4$ . A document bit that agrees with the query contributes  $+a_i$ ; a disagreement contributes  $-a_i$ .

$$S(q, b) = \sum_{i=1}^d q_i b_i = A - 2 \underbrace{\sum_{i=1}^d a_i \mathbf{1}[b_i \neq t_i]}_{\pi_q(b)}. \quad (1)$$

The quantity  $\pi_q(b)$  is the **weighted mismatch penalty**. Maximizing score is exactly the same as minimizing this penalty. The factor two is not a hardware setting: replacing a contribution  $+a_i$  by  $-a_i$  changes the score by  $2a_i$ .

| Document | Code | Mismatching coordinates | Score |
|----------|------|-------------------------|-------|
| 1        | 1010 | none                    | 1.4   |
| 2        | 1011 | 4                       | 1.2   |
| 3        | 1000 | 3                       | 1.0   |
| 4        | 0010 | 1                       | 0.0   |
| 5        | 1110 | 2                       | 0.6   |
| 6        | 0101 | 1, 2, 3, 4              | -1.4  |
| 7        | 1001 | 3, 4                    | 0.8   |
| 8        | 0011 | 1, 4                    | -0.2  |

For example, document 7 pays  $0.2 + 0.1 = 0.3$ , so its score is  $1.4 - 2(0.3) = 0.8$ . The exact global top two are documents 1 and 2. We break equal scores by a stable numeric document ID throughout. This is the internal ID used by the index; an external identifier or URL can be recovered through metadata.

The query gives both a preferred sign and a cost for getting that sign wrong. It does *not* tell us which preferred patterns exist in the database. That distinction drives both branching and key enumeration.

### 1.4 Matryoshka prefixes are useful; they are not sorted magnitudes

A Matryoshka model trains selected prefixes of an embedding to remain useful representations [2]. It does not impose  $|q_1| \geq |q_2| \geq \dots \geq |q_D|$  for every query. A later coordinate may have a larger magnitude than an earlier one. Nor does the name of the training method tell us the numerical recall of the first  $h$  binary signs.

There are three distinct choices:

- Take the first  $h$  coordinates: a fixed prefix that can be indexed in advance.
- Sort all coordinates by  $|q_i|$ : a query-dependent traversal order, useful for bitplanes.
- Use a longer representation for final scoring: this can change the ranking after retrieval.

A hash table built for one fixed prefix does not become a table for the query’s largest-magnitude coordinates just because the query is sorted. The lookup coordinates would have changed.

## 1.5 Parameters and units

The formulas below are not tied to a particular code length, cluster count, word width, or SIMD implementation. Table 1 is the common notation. Additional symbols are introduced where their algorithm first needs them.

**Table 1:** Shared parameters. Counts and byte sizes are different types of quantity.

| Symbol                            | Meaning                                                             | Units             |
|-----------------------------------|---------------------------------------------------------------------|-------------------|
| $N$                               | Number of stored documents                                          | documents         |
| $D$                               | Full embedding dimension                                            | coordinates       |
| $d$                               | Dimension of the binary score in Equation (1)                       | bits / doc        |
| $d_r, d_f$                        | Routing and optional final-reranking dimensions                     | coordinates       |
| $C$                               | Number of disjoint clusters                                         | clusters          |
| $n_j$                             | Physical number of document rows in cluster $j$ ; $\sum_j n_j = N$  | documents         |
| $\mathcal{P}(q), P$               | Probed cluster set and its size                                     | set, count        |
| $e_j$                             | Eligible rows in probed cluster $j$ , after filters; $e_j \leq n_j$ | documents         |
| $N_{\text{sel}}, N_{\text{elig}}$ | $\sum_{j \in \mathcal{P}} n_j$ and $\sum_{j \in \mathcal{P}} e_j$   | documents         |
| $K, R$                            | Final result count and binary candidate-heap capacity, $K \leq R$   | documents         |
| $w, b_w$                          | Bits and bytes per bitmap word; $b_w = w/8$                         | bits, bytes       |
| $g, G$                            | Bits per score-table group and $G = \lceil d/g \rceil$ groups       | bits, groups      |
| $b_s, b_q$                        | Bytes per stored table score and query float                        | bytes             |
| $b_{\text{id}}, b_o$              | Bytes per document ID and offset                                    | bytes             |
| $b_c, b_f$                        | Bytes per routing-centroid and final-vector coordinate              | bytes             |
| $M_{\text{RAM}}$                  | Available physical RAM budget                                       | bytes             |
| $J$                               | Number of simultaneously active queries                             | queries           |
| $c_x$                             | Effective time for a named unit of work $x$                         | seconds/unit      |
| $T_{\text{emb}}, Q_{\text{emb}}$  | Query-embedding latency and its peak scratch RAM                    | seconds,<br>bytes |

Assume positive integer dimensions and counts,  $1 \leq P \leq C$ ,  $1 \leq K \leq R$ , and a word width  $w$  divisible by eight. A selected pool may contain fewer than  $R$  eligible rows, in which case fewer candidates are returned.

Define two widths we will use repeatedly:

$$s_{\text{code}} = b_w \lceil d/w \rceil, \quad W_j = \lceil n_j/w \rceil, \quad s_{\text{mask},j} = b_w W_j. \quad (2)$$

The first describes one packed *document*; the second describes one bitplane across a *cluster*. They have different axes. If a filter leaves only a few eligible rows,  $W_j$  does not shrink unless the data is physically rebuilt or represented differently.

## 1.6 Three kinds of statement in this report

An algebraic identity, such as Equation (1), is exact under its stated score. An operation count is exact for the stated pseudocode or layout, but can change with the implementation. A latency or statistical estimate also needs hardware or distribution assumptions. Measured tables report a particular experiment. Keeping these categories separate is necessary for a fair comparison.

## 2 What is built once, and what every query pays

### 2.1 The public Exa-style starting point

Exa’s public description combines Matryoshka prefixes, binary document codes, float-query scoring through small lookup tables, similarity-based clustering, filtering, and final reranking [1]. Its published example uses particular dimensions and cluster counts; our formulas replace those choices with parameters. The pseudocode here is an explicit reference design, not a claim to reproduce Exa’s private implementation or production latency.

### 2.2 Preprocessing and the common index

A document is encoded before queries arrive. Its code and document ID are placed in its cluster’s contiguous row range. The baseline ownership convention is simple: each document has one packed code and one ID. Cluster offsets identify the ranges. Bitplanes or hash-key postings are additional structures described later.

**Listing 1:** Common preprocessing; executed before serving queries.

```
BuildCommonIndex(documents, model, d, routing_configuration):
    vectors = encode documents with model
    routing = train or choose the cluster assignment rule
    for each document x:
        b = signs of the first d coordinates of vectors[x]
        j = assign x to one cluster
        append (document_id[x], packed(b)) to cluster j
    concatenate cluster rows and record C + 1 offsets
    save centroids if centroid routing is used
    save filter metadata and optional final vectors
    return common index
```

Offline encoding, clustering and sorting are build costs. They are not charged as query latency. Their outputs *are* charged as index storage. Build-time temporary arrays can be large; the serving-RAM formulas below deliberately do not describe the construction peak.

For centroid routing, define

$$M_{\text{base}} = N s_{\text{code}} + N b_{\text{id}} + (C + 1) b_o + C d_r b_c + M_{\text{filter}} + M_{\text{extra}}. \quad (3)$$

$M_{\text{extra}}$  includes explicitly chosen common metadata and optional stored final vectors. For example, fully resident reranking vectors contribute  $N d_f b_f$ . If they are on disk, they still consume storage, but only the resident portion belongs in serving RAM. For sign-prefix routing, omit the centroid term. Do not silently count a second ID array in one method and omit the only ID array in another.

**Example: Stored bytes versus touched bytes**

Suppose an illustrative database has  $N = 10^9$  documents and  $d = 256$  bits. Packed code payload without extra padding is  $10^9 \cdot 256/8 = 32$  GB. With eight-byte IDs, the codes and IDs together need 40 GB before centroids or metadata. Here GB means  $10^9$  bytes. These are example choices for  $N, d, b_{\text{id}}$ , not constants in Equation (3).

**2.3 Step 1: encode and prepare the query****Listing 2:** Query preparation shared by all methods.

```

PrepareQuery(text, model, D, d, d_r):
    full_query = model.encode(text, D)
    q = prepare prefix full_query[0:d] for the chosen score
    routing_query = prepare prefix full_query[0:d_r]
    t[i] = +1 if q[i] >= 0 else -1
    a[i] = abs(q[i])
    return full_query, q, routing_query, t, a

```

The model’s tokenization, neural-network computation and output processing cost  $T_{\text{emb}}(\ell_q, D)$  for query length  $\ell_q$ . This is a measured function, not a universal linear formula in  $D$ . Truncation, optional normalization, signs and magnitudes add  $T_{\text{prep}} = O(D + d + d_r)$  elementary work. Their coefficient can be measured separately. A norm or layer-normalization rule must be the same rule used when preparing the documents.

Query-vector storage is  $O(D + d + d_r)$  floats, with possible views instead of copies. Record the chosen ownership. We denote the actual retained query buffers by  $Q_{\text{vec}}$  and the embedding model’s own scratch peak by  $Q_{\text{emb}}$ .

**2.4 Step 2: select clusters**

With a flat centroid router, score all  $C$  centroids, then retain the best  $P$ . This is distinct from scoring binary documents: centroids are normally floats.

**Listing 3:** A simple flat centroid router with bounded selection memory.

```

Route(q_route, centroids, P):
    nearest = heap with capacity P
    for j in all C clusters:
        route_score = dot(q_route, centroid[j])
        offer (route_score, j) to nearest
    return the P selected cluster IDs

```

Let  $c_{\text{fma}}$  price one centroid coordinate’s multiply-add contribution, and let  $H_r$  be the number of heap comparisons or equivalent selection operations. Then

$$T_{\text{route}} = Cd_r c_{\text{fma}} + H_r c_{\text{sel}}, \quad H_r = O(C \log(P + 1)). \quad (4)$$

A streaming heap needs  $O(P)$  records; materializing all centroid scores needs  $O(C)$  scores instead. Either is valid, but their RAM formulas differ. Hierarchical or approximate routing has its own

measured visited-centroid count; replace  $C$  by that count only when the algorithm actually avoids the other centroids. Its routing recall also needs evaluation.

A sign-prefix router uses  $r$  fixed sign coordinates and at most  $2^r$  addresses. It is not the same as similarity-based centroid clustering. Assigning a document is cheap, but ranking many possible query buckets is not free. One reference router scores all  $C$  bucket keys using  $\lceil r/g \rceil$  lookup terms each and selects  $P$ , giving

$$T_{\text{route,sign}} = T_{\text{table,r}} + C \lceil r/g \rceil c_{\text{lut}} + H_r c_{\text{sel}}.$$

A direct or multi-probe address generator can replace that router, but must count its own key-generation and queue work. When every cluster is probed, routing can simply return all cluster IDs and skip distance-based selection.

## 2.5 Cluster count does not determine the selected document count

Exactly,

$$N_{\text{sel}}(q) = \sum_{j \in \mathcal{P}(q)} n_j. \quad (5)$$

Only under balanced clusters is this approximately  $NP/C$ . An optional load factor  $\lambda(q) = N_{\text{sel}}/(NP/C)$  summarizes deviation for a *given query*; it is not automatically constant as  $P$  changes. For bitplanes, the individual  $n_j$  matter because wider clusters make each node more expensive. A mean load factor does not identify the largest cluster or peak frontier memory.

## 2.6 Step 3: apply optional filters

A filter can return eligible row IDs or a bitmap. Denote its time by  $T_{\text{filter}}$  and its scratch by  $Q_{\text{filter}}$ . For a bitmap intersection over the probed clusters, a reference work count is  $\sum_{j \in \mathcal{P}} W_j$  word visits per input mask. For sorted ID lists, cost depends on list lengths and the intersection method instead.

After filtering,  $e_j$  rows are eligible in cluster  $j$ , so a scan scores  $N_{\text{elig}} = \sum_j e_j$  documents. A dense bitplane still has physical width  $W_j$ . A filter that removes most rows therefore helps scoring immediately, but does not automatically make each bitplane intersection short.

The reference set for recall must obey the *same filter*. Otherwise a correct filtered search is unfairly penalized for not returning ineligible documents.

## 2.7 Step 4: build the float-query score tables

Split the  $d$  document signs into groups of  $g$ . A group has at most  $2^g$  sign patterns. For each query group, precompute its dot product with every possible pattern.

**Listing 4:** The score table. Padding contributes zero in the last group.

```

BuildScoreTable(q, g):
    for group in ceil(len(q) / g) groups:
        for pattern in all g-bit patterns:
            table[group, pattern] = 0
            for coordinate in this group:
                add q[coordinate] * sign(pattern, coordinate)
    return table

ScoreDocument(packed_code, table):
    score = 0
    for group in all groups:
        pattern = extract this group's bits from packed_code
        score += table[group, pattern]
    return score

```

For this straightforward build,

$$G = \lceil d/g \rceil, \quad U_{\text{build}} \leq G2^g g, \quad (6)$$

$$T_{\text{table}} = U_{\text{build}} c_{\text{build}}, \quad Q_{\text{table}} = G2^g b_s, \quad (7)$$

$$T_{\text{score}}(F) = FGc_{\text{lut}}. \quad (8)$$

$c_{\text{lut}}$  includes extracting the pattern, the lookup and accumulation in the chosen kernel. Faster table construction is possible, but it must replace  $U_{\text{build}}$  explicitly. Larger  $g$  reduces the number of lookup terms and increases the table exponentially. Register capacity and cache behavior therefore matter.

#### Example: Two query values and a four-entry table

For the first two values  $(0.7, -0.4)$ , the contributions for patterns 00, 01, 10, 11 are respectively  $-0.3, -1.1, 1.1, 0.3$ . With  $g = 2$ , document 1011 uses the first-group entry 1.1 and the second-group entry 0.1, giving 1.2.

Plain XOR followed by a population count computes *unweighted* Hamming distance. It matches Equation (1) only when all query magnitudes are equal up to a common scale. Replacing weighted scoring by that operation changes the task.

## 2.8 Step 5: retain candidates and optionally rerank

A top- $R$  heap stores the worst retained candidate at its root. Every scored document needs at least a comparison with that root once the heap is full. Let  $H_s(F, R)$  be the actual selection-operation count. A simple worst-case bound is  $O(F \log(R + 1))$ , not zero.

$$T_{\text{select}} = H_s(F, R)c_{\text{heap}}, \quad Q_{\text{results}} \approx R(b_{\text{id}} + b_s) + Q_{\text{heap,overhead}}. \quad (9)$$

For an optional rerank of  $R'$  returned candidates,  $K \leq R' \leq R$ , let

$$T_{\text{finish}}(R') = T_{\text{fetch}}(R', d_f) + R'd_f c_{\text{fine}} + H_f(R', K)c_{\text{fine,sel}} + T_{\text{output}}(K). \quad (10)$$

$T_{\text{fetch}}$  depends on whether vectors are already resident, on SSD, or remote. If  $u_f$  vectors are fetched in a batch, the temporary float-vector buffer is  $u_f d_f b_f$ , not necessarily  $R' d_f b_f$ . The candidate IDs must remain live until fetching and scoring finish.

## 2.9 Shared latency and peak RAM

For sequential stages, define

$$T_{\text{common}} = T_{\text{emb}} + T_{\text{prep}} + T_{\text{route}} + T_{\text{filter}} + T_{\text{table}}. \quad (11)$$

The method-specific candidate work and Equation (10) are added later. For an index-only measurement, explicitly set the embedding and final-stage terms to zero.

Peak scratch RAM is a maximum of simultaneously live buffers, not a sum of every temporary array ever allocated. If embedding, search and reranking do not overlap,

$$Q_{\text{peak}} = \max\{Q_{\text{emb,stage}}, Q_{\text{search,stage}}, Q_{\text{finish,stage}}\}. \quad (12)$$

Each stage term includes buffers retained from previous stages. We use  $Q_{\text{base}}$  for common buffers live during candidate search, including query vectors, score tables, filter masks, routing output, and the candidate heap.

For  $J$  concurrent queries sharing one resident index,

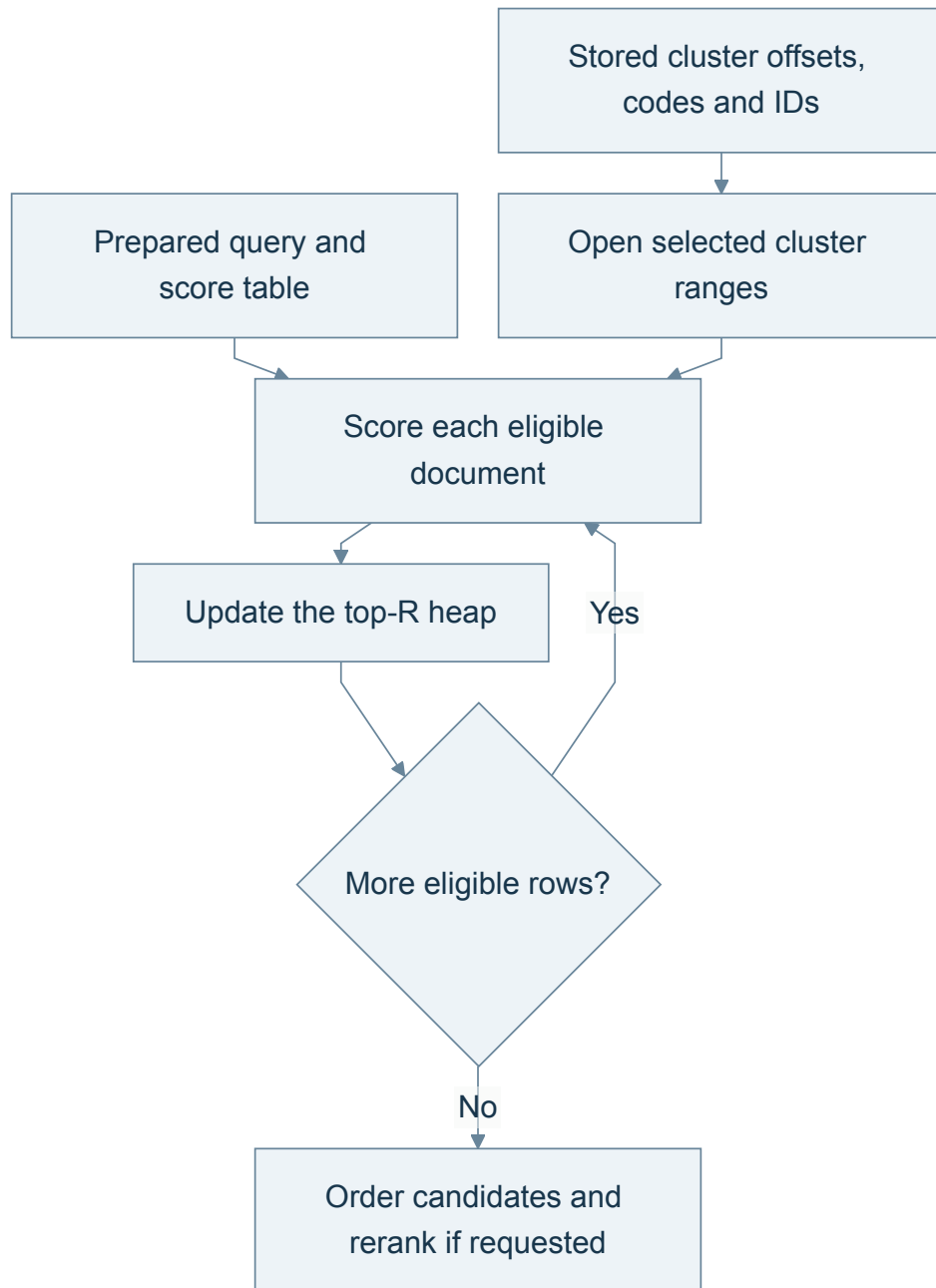
$$\begin{aligned} M_{\text{serving}} &= M_{\text{resident}} + M_{\text{model}} + M_{\text{runtime}} + \max_t \sum_{u \in \mathcal{A}(t)} Q_u(t) \\ &\leq M_{\text{resident}} + M_{\text{model}} + M_{\text{runtime}} + JQ_{\text{peak,max}}. \end{aligned} \quad (13)$$

$M_{\text{model}}$  counts shared resident embedding-model weights when inference is local. An external embedding service contributes latency but no local model-weight allocation.  $M_{\text{runtime}}$  is a chosen reserve or measured runtime overhead. It is not another stored copy of the index. Separate process replicas or device copies must be counted if the actual system has them.

For separate CPU and GPU memories, apply the residency and peak formulas to each memory pool. Count additional device copies if they exist. A throughput speedup does not by itself establish a feasible memory layout.

## 3 Approach A: cluster, then full scan

This is the reference candidate-search method. Once clusters and eligible rows are chosen, it scores every eligible row. There is no approximation *inside* that selected pool. The work is regular and can read packed codes sequentially when filtering does not scatter them.



**Figure 2:** Approach A. The selected pool is smaller than the corpus only because of routing or filters.

### 3.1 Data layout and preprocessing

The common index from Equation (3) is sufficient. Cluster  $j$  occupies the row range between offsets  $j$  and  $j + 1$ . A row contains its packed code, and the corresponding ID array maps the row back to the original document.

**Listing 5:** Candidate search for Approach A.

```
ScanSelectedClusters(q, selected_clusters, eligible, table, R):
    best = empty top-R heap
    for cluster j in selected_clusters:
        for row in eligible rows of j:
            score = ScoreDocument(code[row], table)
            best.offer(score, document_id[row])
    return best in descending score, breaking ties by ID
```

The loop is over  $N_{\text{elig}}$  rows, not necessarily  $NP/C$ . If all clusters are searched and no filter removes documents,  $N_{\text{elig}} = N$ . Empty filtered clusters require no document scoring, although finding that they are empty can still have a cost.

### 3.2 Step-by-step cost

| Step                  | Work                                           | Live temporary data                |
|-----------------------|------------------------------------------------|------------------------------------|
| Locate cluster ranges | $P$ offset/range accesses                      | selected cluster IDs               |
| Visit eligible rows   | $N_{\text{elig}}$ row iterations               | current row/iterator               |
| Score rows            | $N_{\text{elig}}G$ lookup terms                | shared score table,<br>accumulator |
| Keep best $R$         | $H_s(N_{\text{elig}}, R)$ selection operations | top- $R$ heap                      |
| Return candidates     | sort at most $R$ results if needed             | result array or heap<br>storage    |

Writing  $c_{\text{range}}$  for range setup and  $T_{\text{rank}}(R)$  for final candidate ordering,

$$T_{A,\text{candidates}} = P c_{\text{range}} + N_{\text{elig}} G c_{\text{lut}} + H_s(N_{\text{elig}}, R) c_{\text{heap}} + T_{\text{rank}}(R). \quad (14)$$

The general end-to-end latency is

$$T_A = T_{\text{common}} + T_{A,\text{candidates}} + T_{\text{finish}}(R'). \quad (15)$$

For the stated heap implementation, the candidate-stage time complexity is

$$O(P + N_{\text{elig}} \lceil d/g \rceil + N_{\text{elig}} \log(R + 1) + R \log(R + 1)).$$

The lookup term count is not a claim that each term is exactly one CPU instruction.

### 3.3 What is read, and what must remain in RAM?

The packed-code payload read for scoring is

$$V_{A,\text{codes}} = N_{\text{elig}} s_{\text{code}}. \quad (16)$$

ID reads, filter data, table accesses and result-heap accesses are additional traffic. A cache may satisfy many of them. Thus this is a named payload count, not a complete DRAM measurement. Sequential and scattered rows may have different effective  $c_{\text{lut}}$  or  $c_{\text{row}}$ .

Index storage and candidate-stage scratch are

$$M_{A,\text{index}} = M_{\text{base}}, \quad (17)$$

$$Q_{A,\text{search}} = Q_{\text{base}} + Q_{\text{row iterator}} + Q_{\text{scan workspace}}. \quad (18)$$

A streaming scan does not need an  $N$ -element score array. A matrix-based batch implementation may choose to allocate such an array; then that allocation must be added. The same algorithmic ranking can therefore have different peak RAM under different implementations.

### 3.4 A routing miss in the four-bit example

Partition the eight documents from Section 1 into

$$\mathcal{C}_0 = \{1, 3, 4, 6\}, \quad \mathcal{C}_1 = \{2, 5, 7, 8\}.$$

This is an illustrative fixed partition, not a claim about the output of a particular clustering run. Suppose the router opens only  $\mathcal{C}_1$ . A scans scores 1.2, 0.6, 0.8,  $-0.2$  and returns documents 2 and 7. The global top two are 1 and 2, so recall is  $1/2$ . The scan did nothing wrong inside the opened cluster: document 1 was never considered.

Opening both clusters returns documents 1 and 2. This is why a scan can be exact in its selected pool and still have imperfect global recall.

### 3.5 SIMD changes a coefficient, not the score

SIMD means one instruction operates on several data lanes. It can accelerate code extraction, lookup accumulation, and selection. It does not by itself change the number of stored bits. There are two useful ways to model its effect.

**Calibrated model.** Measure the implemented kernel's effective time per work unit,  $c_{\text{lut}}$ , for the relevant dimensions and access pattern. If that rate already includes selection, use a combined  $T_{\text{score+select}}$  and do not add the heap cost a second time.

**Resource model.** If a kernel performs  $U_{\text{op}}$  compute units and causes  $V_{\text{DRAM}}$  bytes of DRAM traffic, compute throughput  $\Gamma$  and memory bandwidth  $\beta$  give the lower bound

$$T_{\text{kernel}} \geq \max\{U_{\text{op}}/\Gamma, V_{\text{DRAM}}/\beta\}. \quad (19)$$

An effective SIMD factor  $s_{\text{simd}}$  can change  $\Gamma$  to  $s_{\text{simd}}\Gamma_0$ . It cannot make the bandwidth term disappear. Random access, synchronization and fixed overhead can put actual time above the bound. Do not add this resource estimate on top of a fitted coefficient that already includes the same work.

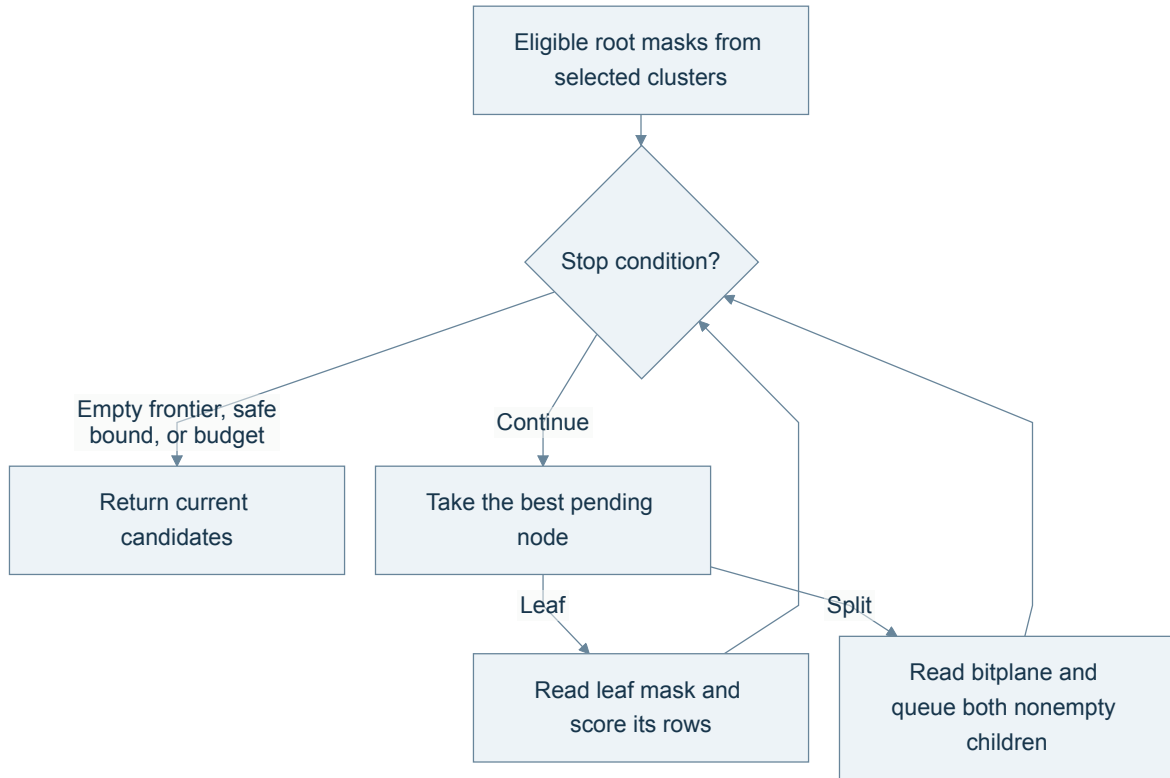
For multiple queries, a float score matrix can be written as  $QB^\top$ , where rows of  $B$  are expanded binary codes. That exposes matrix operations, but expansion, batching delay, device transfers and score-buffer storage have costs. A fast GPU batch is not automatically a fast single-query request.

#### What Approach A guarantees

For a fixed eligible pool, A returns its exact binary top  $R$  under the stated tie rule. With every eligible cluster searched, it has 100% binary recall. With partial routing, its global binary recall is limited by which true neighbors reach the selected pool. No property of the scan restores documents missed by routing, quantization or an earlier filter.

## 4 Approach B: cluster, then inverted bitplanes and branching

Approach B keeps the same document score and common routing stage. Its extra index stores the document bits by coordinate instead of only by row. That makes one sign test applicable to many document positions at once.



**Figure 3:** Approach B. A pending node has a set of active documents and an optimistic score bound.

#### 4.1 What a bitplane stores

For the opened example cluster  $\mathcal{C}_1 = (2, 5, 7, 8)$ , the document codes are 1011, 1110, 1001, 0011. The transposed index is

|            | doc 2 | doc 5 | doc 7 | doc 8 |
|------------|-------|-------|-------|-------|
| bitplane 1 | 1     | 1     | 1     | 0     |
| bitplane 2 | 0     | 1     | 0     | 0     |
| bitplane 3 | 1     | 1     | 0     | 1     |
| bitplane 4 | 1     | 0     | 1     | 1     |

Each row is a bitmap over document positions, not a list of full document vectors. An active bitmap says which document positions still belong to the current node. To require coordinate  $i$  to be 1, intersect the active bitmap with plane  $i$ . To require 0, intersect with its complement, while masking unused padding positions.

**Listing 6:** Build the additional bitplane index.

```
BuildBitplanes(cluster_codes, d, w):
    for each cluster j:
        W[j] = ceil(number_of_rows(j) / w)
        planes[j] = d arrays of W[j] zero words
        for local row u in j:
            for coordinate i in 0 .. d-1:
                if code[j, u, i] is 1:
                    set bit (u mod w) in planes[j, i, floor(u/w)]
    return planes
```

The extra persistent storage is exactly, for this layout,

$$M_{\text{planes}} = db_w \sum_{j=1}^C W_j, \quad M_{B,\text{index}} = M_{\text{base}} + M_{\text{planes}}. \quad (20)$$

Without per-cluster padding this is  $Nd/8$ , a second binary-code-sized payload. For example, ten billion 256-bit codes occupy 320 GB; their bitplanes add approximately another 320 GB, not 32 TB. Padding and other index structures are additional.

#### 4.2 Why the query suggests a traversal direction

At coordinate  $i$ , choosing the query's sign adds no penalty; choosing the opposite sign adds  $a_i = |q_i|$ . Sorting coordinates by descending  $a_i$  tests the largest potential penalties first. For the four-bit query, the order is 1, 2, 3, 4 with weights 0.7, 0.4, 0.2, 0.1.

In  $\mathcal{C}_1$ , matching bit 1 keeps documents 2, 5, 7; the alternative contains document 8 with accumulated penalty 0.7. Matching bit 2 next keeps 2, 7; document 5 moves to another pending node with penalty 0.4. This is a useful order, but neither alternative is automatically safe to discard.

**Example: A greedy path can miss the best document**

Take  $q = (0.6, 0.5, 0.4, 0.3)$  and two documents: 0111 and 1000. The first has penalty 0.6 and score 0.6. The second has penalty 1.2 and score  $-0.6$ . A greedy choice of the matching first bit selects 1000 and misses the better document. The largest individual weight is smaller than the sum of several remaining weights.

**4.3 A bound makes branching safe**

Let a node have accumulated penalty  $p$  on coordinates already tested. Untested coordinates cannot contribute a negative mismatch penalty. Consequently,

$$U(\text{node}) = A - 2p \quad (21)$$

is an upper bound on every active document's final score. If the result heap is full and  $U$  is strictly below its worst retained score, that node cannot improve the result. For equal scores, keep searching unless the ID tie rule is also certified.

The **frontier** is the collection of pending nodes. A best-first search takes the node with the largest upper bound, or equivalently the smallest penalty. The bound is optimistic: it may describe a sign combination that no actual document can achieve. A weak bound can keep many nodes alive even though none ultimately improves the answer.

**Listing 7:** Dense bitplane search across the selected clusters.

```
SearchBitplanes(q, selected_clusters, eligible, table, R, L, B_node):
    order = coordinates sorted by decreasing abs(q[i])
    frontier = empty min-penalty priority queue
    best = empty top-R heap
    for each selected cluster j:
        active = bitmap of eligible rows, with physical width W[j]
        if active is not empty:
            frontier.push(j, depth=0, penalty=0, active)
    visits = 0
    while frontier is not empty:
        if best is full and best_possible(frontier) < best.worst_score:
            break
        if a finite B_node is set and visits == B_node:
            break
        node = frontier.pop_smallest_penalty()
        visits += 1
        if node.count <= L or node.depth == d:
            for word in node.active:                # includes zero words
                for each set bit in word:
                    score its document and offer it to best
            continue
        i = order[node.depth]
        matching = active AND preferred_sign_plane(j, i, q)
        opposite = active AND NOT preferred_sign_plane(j, i, q)
        queue nonempty matching child with the same penalty
        queue nonempty opposite child with penalty + abs(q[i])
    return best
```

$L$  is the leaf-size threshold and  $B_{\text{node}}$  the total node budget across all probes. An unlimited budget permits exact termination by bounds; a finite budget can stop before the remaining nodes are safely ruled out. A random exploration probability changes which pending node is visited next. It is not a direct formula for recall or a synonym for the number of nodes.

#### 4.4 Count split visits, leaf visits, and scored documents separately

For each selected cluster, let

$s_j$  = visited split nodes,  $\ell_j$  = visited leaf nodes,  $f_j$  = documents fully scored.

Define  $V = \sum_j (s_j + \ell_j)$  and  $F = \sum_j f_j$ . For a disjoint traversal without rescoring,  $F \leq N_{\text{elig}}$ . A finite node budget gives  $V \leq B_{\text{node}}$ , but does not determine  $F$ . The two principal bitmap counts are

$$U_{\text{split}} = \sum_j s_j W_j, \quad U_{\text{leaf}} = \sum_j \ell_j W_j. \quad (22)$$

A split word in the reference implementation reads an active word and a plane word, produces two child words, and counts matches. It includes two ANDs, a population count, complements and an accumulation. Allocating zero-filled child arrays can add two initialization writes. A leaf still scans all words in its active bitmap before enumerating set bits.

With separate effective coefficients,

$$T_{B,\text{candidates}} = T_{\text{order}} + T_{\text{roots}} + U_{\text{split}} c_{\text{split}} + U_{\text{leaf}} c_{\text{leafword}} + V c_{\text{node}} + FG c_{\text{lut,leaf}} + H_s(F, R) c_{\text{cheap}} + T_{\text{rank}}(R), \quad (23)$$

$$T_B = T_{\text{common}} + T_{B,\text{candidates}} + T_{\text{finish}}(R'). \quad (24)$$

Here  $T_{\text{order}} = O(d \log d)$  and  $T_{\text{roots}} = O(\sum_j W_j)$  for dense root construction.  $c_{\text{node}}$  may be replaced by explicit queue-comparison, mask-allocation and copy terms. Use one convention consistently; do not charge an allocation twice.

#### 4.5 The full-width cost is the important hidden term

##### Example: Few scores do not imply little work

In the saved one-million-document run,  $w = 64$  gives  $W = 15,625$ . A node budget of 32,768 produced a mean 22,557.88 split visits and 10,210.12 leaf visits. Therefore

$$U_{\text{split}} = 352,466,875, \quad U_{\text{leaf}} = 159,533,125.$$

Their sum is 512 million word-loop visits. About 163,766 documents were fully scored. With  $g = 4$ , that is 10.48 million score-table terms, versus 64 million terms for the scan. The reduction in scoring was real, but it came with hundreds of millions of bitmap visits. Word visits and score-table terms are not equal-cost instructions.

If a tree were fully explored, perfectly balanced, and every internal node had two nonempty children, then with  $L$  documents per leaf there would be roughly  $n_j/L$  leaves and one fewer split. This gives a dense bitmap term of order

$$O\left(\sum_j \frac{n_j^2}{Lw}\right). \quad (25)$$

This is an explanatory idealization, not a predictor for real embeddings. Fixed-bit splits can have only one nonempty child. A finite traversal also leaves unvisited nodes. Thus the actual split count need not be half the visited-node count.

A single greedy path visits only about  $\log_2(n_j/L)$  levels in a balanced example, but still pays a full  $W_j$  at each level and has no general recall guarantee. In the broad worst case, a fixed-coordinate trie can contain  $O(n_j d)$  nonempty prefix nodes, making dense traversal work as large as  $O(d \sum_j n_j^2/w)$  before scoring and queue costs. Growing  $N$  alone does not guarantee that branching will overtake a scan.

#### 4.6 Peak query RAM: the pending masks matter

Let  $\mathcal{F}(t)$  be the frontier at time  $t$ , and let  $j(v)$  identify a node's cluster. If a node record needs  $b_{\text{node}}$  bytes and bitmap capacity is fully allocated,

$$Q_{B,\text{frontier}}(t) = \sum_{v \in \mathcal{F}(t)} (b_{\text{node}} + b_w W_{j(v)}). \quad (26)$$

Capacity growth and allocator overhead can be represented by a factor  $\gamma_B \geq 1$  or, preferably, by measured capacities. During a split, the parent and newly allocated children can coexist. A conservative candidate-stage expression is

$$Q_{B,\text{search}} = Q_{\text{base}} + Q_{\text{order}} + \max_t [\gamma_B Q_{B,\text{frontier}}(t) + Q_{\text{current}}(t) + Q_{\text{children}}(t)]. \quad (27)$$

For nonempty, disjoint pending sets, frontier count is at most  $N_{\text{elig}}$ . Starting with at most  $P$  roots, each processed binary split can increase that count by at most one, so a finite budget also gives  $|\mathcal{F}| \leq P + B_{\text{node}}$ . Multiplying that bound by the largest possible mask is conservative and can be very loose. It should not be reported as measured RAM.

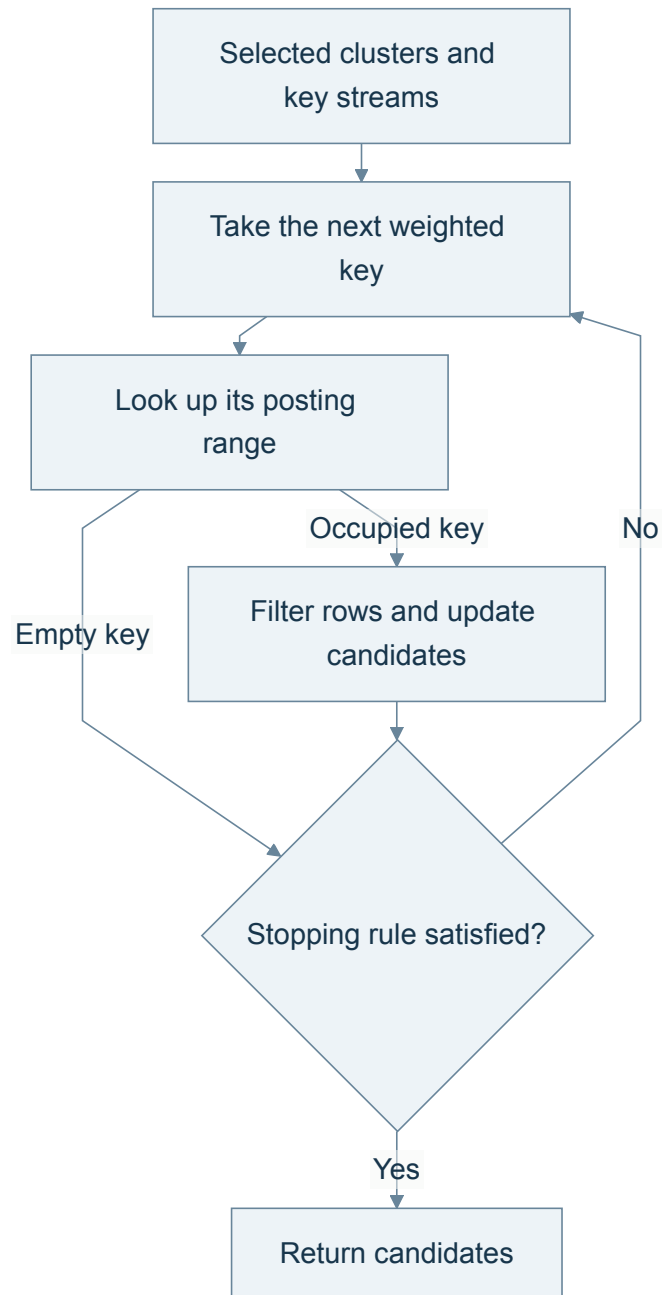
##### When Approach B can help

B can help when it avoids enough full scoring to repay the bitmap, queue and allocation work. Useful conditions include strong early query penalties, small physical bitmaps, tight score bounds and a modest live frontier. Unlimited search is exact inside the selected pool under valid bounds and ties. Budget-limited search needs measured recall; its budget is not itself an accuracy percentage.

### 5 Approach C: cluster, then walk backwards from the ideal key

Approach C changes the lookup structure. Instead of repeatedly intersecting document masks, it asks: *which stored documents have this particular key?* Start from the signs the query prefers, then consider other keys in increasing mismatch penalty.

The primary method here still starts with the same selected clusters. A global hash table is the special case  $C = P = 1$ . We do not silently switch between clustered and global formulas.



**Figure 4:** Approach C. Each cluster has a key directory and postings. Empty lookups still consume work.

## 5.1 Full keys and short keys are different choices

Let  $I_h$  be the fixed set of  $h$  lookup coordinates, often the first  $h$  positions. A **full key** has  $h = d$ . Its weighted penalty determines the complete binary score. A **short key** has  $h < d$ . It ranks a prefix, then the full  $d$ -bit score ranks the retrieved candidates.

A full-key search can be exact if it enumerates keys in true query-weighted order and finishes the result-boundary tie. A short-key candidate quota does not have that guarantee. It can still work well, and can also be made exact with a valid full-score stopping bound. These are properties of the stopping rule, not of the name “walk backwards.”

## 5.2 The data structure behind a key lookup

For each cluster, a directory maps a key to a posting range containing the local row references of documents with that key. There may be many documents per key. Finding a directory entry without finding its document rows is not enough to produce results.

**Listing 8:** Build a clustered key index.

```
BuildKeyIndex(cluster_codes, lookup_coordinates):
    for each cluster j:
        groups = group local row references by their lookup key
        concatenate the groups into a posting array
        for each occupied key:
            directory[j][key] = (posting_start, posting_count)
    return directories, postings
```

A **dense directory** has an offset for every possible key. A **sparse hash table** only stores occupied keys but adds key, slot and load-factor overhead. A sorted-key array is another option: it can use less overhead, but a lookup uses comparisons or another search structure instead of an assumed constant-time hash access.

## 5.3 Do not index fixed routing bits as if they were still free

If sign-prefix routing fixes some lookup coordinates inside cluster  $j$ , those coordinates cannot vary within it. Let  $r_j$  be the number of lookup coordinates fixed by that cluster, and let

$$h_j = h - r_j, \quad U_j \leq \min(n_j, 2^{h_j}). \quad (28)$$

$U_j$  is the number of occupied free keys. For first- $r$ -bit routing and a first- $h$ -bit lookup with  $h \geq r$ ,  $h_j = h - r$ . The fixed coordinates add a *constant* penalty for that cluster; they do not create  $2^r$  additional keys inside it. Ordinary k-means clusters do not generally fix any sign coordinate, so this simplification does not follow from their cluster ID.

### Example: Two fixed bits and a two-bit local key

Suppose a four-bit code is routed by its first two bits. Inside cluster 10, codes 1010, 1011, 1000, 1001 need only the local keys 10, 11, 00, 01. A lookup for the full prefix must include the cluster’s constant routing penalty when it is compared with a different cluster’s stream. The number of local possibilities is four, not sixteen.



Both child costs are no smaller than the parent cost. Every nonempty subset has one parent: if the coordinate before its largest coordinate is also present, remove the largest one; otherwise replace the largest coordinate by its predecessor. This explains why the construction visits every subset without needing a duplicate-state hash set.

A state must store enough information to construct the key. A full bitmask representation costs  $b_w \lceil h_j/w \rceil$  bytes per state, plus its penalty, last coordinate and queue metadata. Copying that mask into children is work. A persistent representation can change this cost, but it then needs its own traversal and memory accounting.

**Example: The full-key order can be accurate even when a greedy branch is not**

For  $q = (0.6, 0.5, 0.4, 0.3)$ , the ideal code is 1111. The first penalties include 0, 0.3, 0.4, 0.5, 0.6, 0.7, and so on. The code 0111 is found by flipping coordinate 1 at penalty 0.6. The code 1000 has penalty 1.2 and comes later. Complete weighted enumeration finds the better document first, even though it disagrees on the query's largest coordinate. It may have to check several empty keys before that hit.

## 5.6 Lookup and stopping within selected clusters

For short keys, a simple implementation allocates candidate targets  $m_j$  to probed clusters. A more flexible implementation merges their next-prefix-key scores and draws candidates from the best available stream. Whichever policy is used must be a parameter of the experiment; a per-cluster target and a global target are not interchangeable.

**Listing 10:** Short-key candidate search with explicit per-cluster targets.

```
SearchShortKeys(q, selected_clusters, h, targets m[j], R):
    candidates = empty collection
    for each selected cluster j:
        stream = SortedFlipSets(free lookup weights in j)
        found = 0
        for (flip_set, variable_penalty) in stream:
            key = ideal_free_key(q, j) XOR flip_set
            rows = directory[j].lookup(key)      # may be empty
            keep eligible rows from this posting range
            found += number of kept rows
            append those rows to candidates
            if found >= m[j] and final prefix-penalty ties are finished:
                break
    score every candidate with the full d-bit score
    keep the best R with the same score/ID rule as A
    return best R
```

A full-key exact version instead merges the cluster streams by their *total full-key* penalty, including fixed-sign penalties. Once at least  $R$  documents have been found, continue through every key that can tie or exceed the  $R$ th document. This produces the exact binary top  $R$  within the selected pool. If unprobed clusters remain, global exactness additionally requires their score bounds to be ruled out or those clusters to be searched.

A short key can also stop exactly. Let  $T_h(b)$  be the prefix score and  $A_{\text{tail}} = \sum_{i \notin I_h} |q_i|$ . Every

document in an unvisited prefix key satisfies

$$S(q, b) \leq T_h(b) + A_{\text{tail}}. \quad (31)$$

If the best unvisited prefix upper bound is strictly below the current worst top- $R$  score, the result is certified. If not, continue even when a nominal candidate quota has been reached. A larger tail makes this bound less selective. Fixed routing-coordinate contributions must be included consistently; no coordinate may be counted twice.

### 5.7 Do we need to rescore a document whose full key is known?

Not always. With a full key and the true query weights, its complete score is already  $A - 2\pi_q(\text{key})$ . Every document in that key’s posting range has that same binary score. The search can offer their IDs to the result heap without unpacking and rescoring each code. For a short key, the known prefix contribution can similarly be reused while evaluating only the remaining coordinates.

To make that implementation choice visible, let  $G_{C,j}$  be the number of score-table groups actually evaluated per candidate in cluster  $j$ . The simple pseudocode above has  $G_{C,j} = G$ . A full-key implementation reusing its exact key score can have  $G_{C,j} = 0$ . A tail-only scorer can have approximately  $\lceil d_{\text{remaining}}/g \rceil$  groups, with alignment and fixed routing coordinates handled explicitly. It still pays for postings, key-score arithmetic, selection, and any later float reranking. This optimization changes the cost; it is not a recall loss.

The reference storage formula retains the common packed rows. A specialized full-key layout can also avoid a redundant per-document code copy by using its stored full keys, provided its ID/posting and key ownership are redesigned and counted. Equation (30) is one explicit layout, not a universal lower bound on every possible key index.

### 5.8 If “work upwards” means removing a constraint

There is another reasonable interpretation of the idea. Instead of flipping a bit and probing one more exact key, remove a bit constraint. For example,  $101*$  means either  $1010$  or  $1011$ . That is a set of keys. A plain exact-key hash table does not answer this wildcard request in one ordinary lookup.

A fixed-order **prefix trie** stores paths by successive key coordinates. A node identifies an occupied prefix. If document rows are ordered by key within a cluster, each node can store the start and end of a contiguous posting range. Moving to its parent relaxes the last fixed coordinate. It does not require copying every descendant’s posting list into every ancestor.

**Example: Relaxing the key inside the example cluster**

For  $\mathcal{C}_1 = \{2, 5, 7, 8\}$  and ideal key 1010:

| Constraint | Matching documents | Newly visited documents |
|------------|--------------------|-------------------------|
| 1010       | none               | none                    |
| 101*       | 2                  | 2                       |
| 10**       | 2, 7               | 7                       |
| 1***       | 2, 5, 7            | 5                       |
| ****       | 2, 5, 7, 8         | 8                       |

The parent range contains the previous range. Scan only the newly added sibling ranges if the implementation can retain their boundaries. Otherwise repeated scans must be counted again. In this particular query the last coordinate is the cheapest one to relax. That is not true for every query.

**Listing 11:** A fixed-prefix relaxation variant of Approach C.

```

SearchByRelaxingPrefix(q, query_key, trie, table, R, candidate_target):
    path = follow query_key as far as the stored trie permits
    previous_range = empty
    best = empty top-R heap
    found = 0
    for node on that path, from deepest node toward the root:
        new_ranges = node.posting_range minus previous_range
        for eligible row in new_ranges:
            score its code and offer it to best
            found += 1
        previous_range = node.posting_range
        if found >= candidate_target:
            return best                                # approximate stopping rule
    return best

```

An exact version may stop early only after bounding *every* unvisited region against the current full-score heap. Simply finding enough IDs in a relaxed prefix is not such a proof. The remaining score can still change the ordering.

Let  $Z_j$  be the number of stored trie nodes and  $b_{\text{trie}}$  their record size, including child links or equivalent navigation and range offsets. A simple uncompressed trie satisfies

$$Z_j \leq 1 + \sum_{u=1}^{h_j} \min(n_j, 2^u).$$

Its directory term replaces the hash-directory term by  $\sum_j Z_j b_{\text{trie}}$ . Path compression can reduce the actual node count. With range references, document postings are stored once; copying a posting list into every prefix node would instead add repeated references and can cost  $O(Nh)$  reference storage.

For  $u_j$  ancestor/range visits and  $F_j$  total row visits, a reference cost is

$$T_{C,\text{trie}} = T_{\text{keyprep}} + \sum_j [h_j c_{\text{path}} + u_j c_{\text{range}} + F_j c_{\text{post}} + F_j G_{C,j} c_{\text{lut,gather}}] + T_{\text{select/rank}}. \quad (32)$$

A saved navigation path uses  $O(h_j)$  records per active stream. Add the candidate heap, gather buffers, and common query RAM. Here  $F_j$  must count repeated rows if they are revisited. The same common pipeline and final rerank terms are added for end-to-end latency.

A fixed trie can drop its last prefix coordinate cheaply. Dropping an arbitrary query-selected low-magnitude coordinate can produce several disjoint ranges, so it does not have the same constant-cost parent operation. A query-adaptive bit order requires additional traversal or a different index. This is the main difference between a prebuilt fixed trie and the query-ordered bitplane traversal.

## 5.9 Count keys, postings, scores and enumeration states

Let  $H_j$  be the number of keys attempted in cluster  $j$ , including empty keys. Let  $F_j$  be the number of eligible posting rows retrieved and scored. Let  $E_j$  be the number of sorted-subset heap comparisons,  $C_j^{\text{copy}}$  the number of key-word copies, and let  $Q_j^{\text{max}}$  be the largest enumeration-queue size. Define  $F = \sum_j F_j$ .

For the two-child stream,  $E_j = O(H_j \log(Q_j^{\text{max}} + 1))$  comparison work, and  $C_j^{\text{copy}} = O(H_j \lceil h_j/w \rceil)$  for full-mask copies. Hashing a key also depends on its length and table layout. An effective  $c_{\text{lookup}}(h_j, \alpha)$  should price the emitted-key iteration outside heap comparisons and child-mask copies: key construction, fixed state bookkeeping, the directory lookup, and obtaining the current key score or bound. This includes cache effects and collisions, rather than assume one CPU operation.

$$T_{C,\text{candidates}} = T_{\text{keyprep}} + \sum_{j \in \mathcal{P}} \left[ E_j c_{\text{enum}} + C_j^{\text{copy}} c_{\text{copy}} + H_j c_{\text{lookup}}(h_j, \alpha) + F_j c_{\text{post}} \right] \\ + \sum_j F_j G_{C,j} c_{\text{lut,gather}} + H_s(F, R) c_{\text{cheap}} + T_{\text{rank}}(R), \quad (33)$$

$$T_C = T_{\text{common}} + T_{C,\text{candidates}} + T_{\text{finish}}(R'). \quad (34)$$

$T_{\text{keyprep}}$  includes sorting lookup weights, initializing streams and constructing query keys. For  $P$  different free-coordinate sets, a straightforward upper count is  $O(\sum_j h_j \log(h_j + 1))$ . A global merge adds its own  $P$ -stream selection work, typically  $O((\sum_j H_j) \log(P + 1))$ ; include it when that variant is used.

The hash directory can be built before the query, but query-specific weighted order depends on  $q$ . Precomputing a hash table does not precompute this order for every float query.

### 5.10 Query RAM while enumerating

Let  $b_{\text{enum},j}$  include a key mask, penalty and queue metadata. For simultaneously active streams, the candidate-search scratch is

$$Q_{C,\text{search}} = Q_{\text{base}} + \max_t \left[ \sum_j \gamma_C Q_j(t) b_{\text{enum},j} + Q_{\text{stream merge}}(t) + Q_{\text{posting buffer}}(t) + Q_{\text{gather}}(t) \right]. \quad (35)$$

A sequential per-cluster implementation can reuse enumeration memory, replacing the sum of live stream sizes by the maximum live cluster stream. A global full-score stream may need all selected streams active. These are different execution choices.

Postings can be streamed into a top- $R$  heap; storing all  $F$  retrieved IDs is not mandatory. If they are materialized for batch reranking, add  $Fb_{\text{ref}}$  or the actual bounded batch size. A sparse dictionary's small stored size does not bound the size of a query's enumeration queue. With the two-child stream, the queue can grow by at most one per emitted state, but  $H_j$  itself can be enormous.

#### What Approach C does and does not lose

Full-key enumeration in true penalty order can have 100% binary recall. Its difficulty can be time or queue memory, especially when most keys are empty. Short-key search is a different tradeoff: it can collect useful candidates cheaply, but a fixed quota is not a recall guarantee. It becomes exact if a valid full-score bound proves that no unseen candidate can improve the result, or if it eventually scores every eligible document.

## 6 Deriving recall without assuming the answer

Suppose a search returns 80 documents from a database of 256. That tells us how much scoring work remains. To calculate recall, we need to know how many of the nearest documents are among those 80. We will start with a small random-bit model where both quantities can be derived, then examine what changes with query weights and clusters.

### 6.1 The exact measured definition

Let  $\mathcal{E}(q)$  be the eligible document set after the intended filter, across the whole database. Let  $\mathcal{G}_K(q)$  be its exact top  $K$  under the chosen reference score, and let  $\mathcal{R}_K(q)$  be the returned results. Then

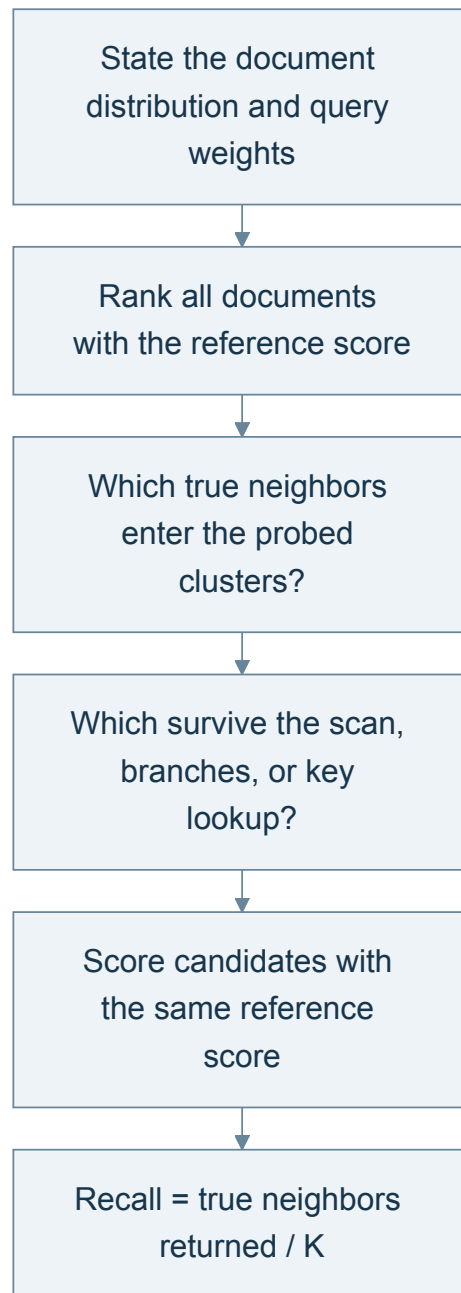
$$\text{Recall@ } K(q) = \frac{|\mathcal{G}_K(q) \cap \mathcal{R}_K(q)|}{K}. \quad (36)$$

Use the same tie rule in both sets. Average this quantity over queries. Repeating one query for timing does not create new independent quality observations.

If a candidate set  $\mathcal{D}(q)$  is reranked with the *same exact reference score*, every true top- $K$  document in that set survives into the final top  $K$ . Therefore

$$\text{Recall@ } K(q) = \frac{|\mathcal{G}_K(q) \cap \mathcal{D}(q)|}{K}. \quad (37)$$

A different reranking score breaks that identity. In particular, exact binary candidates may still miss the true float-vector top  $K$  or the documents relevant to a human question.



**Figure 5:** Follow the same true neighbors through every stage. The denominator stays K throughout the recall calculation.

## 6.2 A small model we can calculate all the way through

Suppose a document has  $d$  bits. We sample  $N$  documents independently, and each bit is independently zero or one with probability  $1/2$ . Duplicate codes are allowed: two different documents can have the same bits. Their row IDs settle score ties, and those IDs are assigned independently of the sampled codes.

For now, every query coordinate has the same magnitude. A document's score therefore depends only on how many bits disagree with the query. We can write the query's desired pattern as all zeros without changing any probabilities. The number of mismatches is called the *Hamming distance*. A smaller distance means a better score.

The search checks  $h$  fixed coordinates and accepts a document if at most  $t$  of them disagree. It then scores every accepted document using all  $d$  bits. For example, with  $h = 4$  and  $t = 1$ , it opens the keys 0000, 0001, 0010, 0100, and 1000. This is a precise candidate rule. It does not stop halfway through a key's documents when a quota is reached.

These assumptions describe a teaching model. Equal weights make counting possible by hand; independent bits make every pattern equally likely. We will remove those assumptions later. There are no clusters or eligibility filters in this first calculation.

## 6.3 Start with one document, then find the nearest of many

Let  $Z$  be one random document's distance from the query. To have exactly  $m$  errors, choose which  $m$  of the  $d$  positions are wrong. There are  $\binom{d}{m}$  choices, out of  $2^d$  equally likely codes. Thus

$$\Pr(Z = m) = \frac{\binom{d}{m}}{2^d}, \quad F_d(m) = \Pr(Z \leq m) = \frac{1}{2^d} \sum_{j=0}^m \binom{d}{j}. \quad (38)$$

This is the binomial distribution with parameters  $d$  and  $1/2$ . The function  $F_d$  adds up the probability of every distance through  $m$ . Such a running probability is called a cumulative distribution function, or CDF. Define  $F_d(-1) = 0$ .

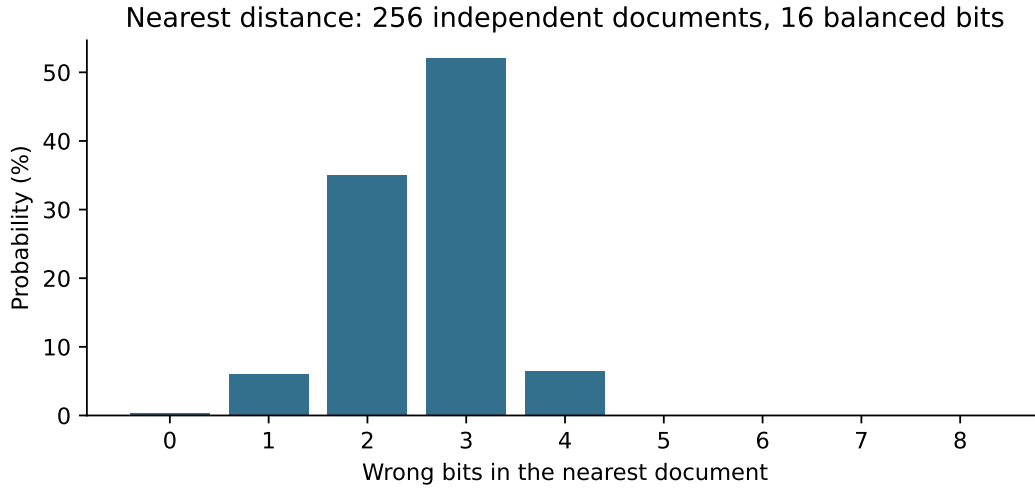
Now let  $M$  be the distance of the nearest document among all  $N$ . If one document has probability  $1 - F_d(m)$  of being farther than  $m$ , independence gives probability  $[1 - F_d(m)]^N$  that *all* documents are farther away. Subtracting from one gives

$$\Pr(M \leq m) = 1 - [1 - F_d(m)]^N. \quad (39)$$

To get exactly distance  $m$ , subtract the probability of a smaller distance:

$$\Pr(M = m) = [1 - F_d(m-1)]^N - [1 - F_d(m)]^N. \quad (40)$$

The distinction matters. One random document usually has many wrong bits. The nearest of hundreds of documents is selected precisely because it has unusually few. We must use its selected distribution when calculating recall.



**Figure 6:** With 256 random 16-bit documents, the nearest distance is usually two or three. Assuming it is always two changes the recall calculation.

#### 6.4 Which error positions survive a short-key search?

Suppose the nearest document has exactly  $m$  wrong bits. Under our uniform model, every set of  $m$  error positions is equally likely. Let  $A$  count how many of those errors fall in the  $h$  lookup positions. To have exactly  $a$  lookup errors, choose  $a$  positions from the lookup bits and  $m - a$  from the other bits. Divide by all possible placements of the  $m$  errors:

$$\Pr(A = a \mid M = m) = \frac{\binom{h}{a} \binom{d-h}{m-a}}{\binom{d}{m}}. \quad (41)$$

This is a hypergeometric distribution. It counts positions drawn from a fixed number of errors. The error count is already fixed here, so treating these positions as independent trials would change the model. A binomial coefficient is zero when its lower argument is negative or larger than its upper argument.

The search accepts at most  $t$  lookup errors. Add the allowed placements:

$$g_{h,t}(m) = \Pr(A \leq t \mid M = m) = \frac{\sum_{a=0}^{\min(h,t)} \binom{h}{a} \binom{d-h}{m-a}}{\binom{d}{m}}. \quad (42)$$

Conditioning on the nearest distance leaves its error positions uniform because the score uses only the total number of errors and the ID tie rule does not favor a particular pattern. This is one place where the equal-weight assumption does real work.

##### Example: Two wrong bits in a 16-bit document

Take  $d = 16$ ,  $h = 4$ , and suppose  $m = 2$ . There are  $\binom{16}{2} = 120$  ways to place the errors. If the first four bits must match perfectly, both errors must be in the other twelve bits:  $\binom{12}{2} = 66$  placements survive. The conditional recall is  $66/120 = 55\%$ .

Allow one error in the first four bits. Another  $4 \times 12 = 48$  placements now survive, giving  $(66 + 48)/120 = 95\%$ . That 95% answer is correct *given two total errors*. It has not yet accounted for the possibility that the nearest document has a different distance.

## 6.5 Combine the two probabilities to obtain recall

For top one, recall is one if the true nearest document is accepted and zero otherwise. Its expected value is therefore its probability of being accepted. For each possible nearest distance, multiply the probability of that distance by the probability of passing the prefix check. Add the results:

$$\mathbb{E}[\text{Recall@1}] = \sum_{m=0}^d \Pr(M = m) g_{h,t}(m). \quad (43)$$

For  $N = 256$ ,  $d = 16$ , and  $h = 4$ , this produces the table below. The candidate counts are derived in the next subsection.

| Allowed errors $t$ | Keys opened | Expected documents | Expected Recall@1 |
|--------------------|-------------|--------------------|-------------------|
| 0                  | 1           | 16                 | 46.40%            |
| 1                  | 5           | 80                 | 89.60%            |
| 2                  | 11          | 176                | 99.45%            |
| 4                  | 16          | 256                | 100.00%           |

For one allowed lookup error, the result is 89.60%, rather than the conditional 95% above. In this example, the nearest distance is two about 35.05% of the time and three about 52.05% of the time. A three-error neighbor is more likely to put too many errors in the lookup bits. Equation (43) includes both cases, along with every other possible distance.

**Listing 12:** Calculate expected top-one recall under the random-bit assumptions.

```
ExpectedRecall(N, d, h, allowed_errors):
    recall = 0
    for m in 0 .. d:
        nearest_probability = (1 - F_d(m-1))^N - (1 - F_d(m))^N
        passing_placements = 0
        for a in 0 .. min(h, allowed_errors):
            passing_placements += choose(h, a) * choose(d-h, m-a)
        survival = passing_placements / choose(d, m)
        recall += nearest_probability * survival
    return recall
```

## 6.6 Count the work separately from the recall

To open all  $h$ -bit keys through radius  $t$ , we try

$$H(h, t) = \sum_{a=0}^t \binom{h}{a}, \quad 0 \leq t \leq h. \quad (44)$$

Each random document has probability  $p_{\text{pass}} = H(h, t)/2^h$  of belonging to one of those keys. If  $F$  counts accepted documents, then

$$F \sim \text{Binomial}(N, p_{\text{pass}}), \quad \mathbb{E}[F] = Np_{\text{pass}}, \quad \text{Var}(F) = Np_{\text{pass}}(1 - p_{\text{pass}}). \quad (45)$$

For  $t = 1$ , five of sixteen keys are opened. The expected candidate count is  $256 \times 5/16 = 80$ , with standard deviation  $\sqrt{55} \approx 7.42$  documents. A particular corpus need not return exactly 80.

The 31.25% candidate fraction also differs from the 89.60% recall: good documents are more likely than arbitrary documents to pass this filter.

For  $C$ ,  $H$  counts lookup attempts, including attempts that return nothing. The documents in those keys supply the posting and scoring work. A short-key query that scores all accepted rows might have expected time

$$\mathbb{E}[T_C] = T_{\text{prepare}} + Hc_{\text{lookup}} + \mathbb{E}[F](c_{\text{post}} + c_{\text{score}}) + \mathbb{E}[T_{\text{select}}],$$

with enumeration and routing added when the implementation performs them. Selection time can depend nonlinearly on the returned rows; substituting  $\mathbb{E}[F]$  into every time term is another approximation. Use the full cost model in Equation (79) for the actual layout.

Storing every candidate ID at once takes  $Fb_{\text{id}}$  bytes. Streaming postings through a bounded top- $K$  heap can avoid that particular full candidate array. Neither choice removes the index itself, and an expected candidate count is not a peak-RAM guarantee.

## 6.7 What changes when we ask for ten or one hundred results?

For top  $K$ , we need the distances of all  $K$  selected rows. Some may have zero errors, some one, and others more. We need the expected number of selected rows at each distance. Assume  $1 \leq K \leq N$ .

Let  $X_m$  count corpus documents whose distance is at most  $m$ . Since every document independently has probability  $F_d(m)$  of meeting that condition,

$$X_m \sim \text{Binomial}(N, F_d(m)).$$

Exactly  $\min(K, X_m)$  of the true top- $K$  rows lie within that radius. There cannot be more than  $K$ , and when fewer than  $K$  documents qualify, every qualifying document belongs to the top  $K$ . Define

$$G_K(p) = \mathbb{E}[\min(K, X)], \quad X \sim \text{Binomial}(N, p). \quad (46)$$

Then the expected number of top- $K$  rows at exactly distance  $m$  is

$$w_m = G_K(F_d(m)) - G_K(F_d(m-1)).$$

The same placement calculation applies to each row at that distance. Therefore

$$\boxed{\mathbb{E}[\text{Recall@ } K] = \frac{1}{K} \sum_{m=0}^d w_m g_{h,t}(m).} \quad (47)$$

This includes tied distances and duplicate codes under our ID rule. For  $K = 1$ ,  $G_1(p)$  is  $1 - (1-p)^N$ , so it reduces to Equation (43). For  $K = N$ , every document is in the reference set and recall becomes the candidate fraction. Opening every key gives recall one for every  $K$ .

The appendix gives a numerically useful way to evaluate  $G_K$ . Notice what this formula does not assume: it never replaces the changing  $K$ -th distance with one fixed guessed radius. It does assume a fixed prefix acceptance rule. Stopping after the first  $R$  postings is a different rule whose boundary depends on the sampled corpus.

## 6.8 Separate routing loss from candidate-search loss

Let  $\mathcal{U}(q)$  be the eligible pool in the probed clusters. Define

$$r_{\text{route}}(q) = \frac{|\mathcal{G}_K(q) \cap \mathcal{U}(q)|}{K}, \quad r_{\text{local}}(q) = \frac{|\mathcal{G}_K(q) \cap \mathcal{D}(q)|}{|\mathcal{G}_K(q) \cap \mathcal{U}(q)|}.$$

When the denominator is positive and  $\mathcal{D} \subseteq \mathcal{U}$ ,

$$\text{Recall@ } K(q) = r_{\text{route}}(q)r_{\text{local}}(q). \quad (48)$$

If no true top- $K$  result enters the selected pool, total recall is zero. This factorization is exact when both factors refer to the *same true global neighbors*. However, multiplying an average routing recall by an average local recall is generally wrong:

$$\mathbb{E}[r_{\text{route}}r_{\text{local}}] = \mathbb{E}[r_{\text{route}}]\mathbb{E}[r_{\text{local}}] + \text{Cov}(r_{\text{route}}, r_{\text{local}}).$$

Nor is recall against a cluster's own local top  $K$  automatically the same as recall for the particular global neighbors routed into it. That distinction matters when transferring the flat-pool branch measurements into a clustered model.

For A, local binary search is exact. For unlimited B with valid bounds, it is exact as well. For full-key C with complete weighted ordering, it is exact. Their global recall can still be limited by routing. Finite node budgets and short-key quotas add a second possible loss.

## 6.9 Apply the candidate rule to the three search methods

The probability calculation describes a set of documents. Several algorithms can reach that same set. If they score those documents with the same full score and use the same tie rule, they have the same recall. Their time and memory costs can still be very different.

**A: open clusters and scan their documents.** For the random-bit example, use the first  $h$  bits as the cluster key. There are  $C = 2^h$  possible clusters. Probing all keys through radius  $t$  gives  $P = H(h, t)$  clusters and exactly the candidate rule we just calculated. With  $h = 4$  and  $t = 1$ , scanning five clusters returns an expected 80 documents and has expected top-one recall 89.60%. The ratio  $P/C = 5/16$  describes the expected fraction of documents read; it is not the nearest-document recall.

A k-means router uses distances to learned centroids instead of counting prefix errors. For each true neighbor, let  $J$  be the rank of its assigned cluster in the query's routing order. Probing the first  $P$  clusters retains that neighbor when  $J \leq P$ . Routing recall is therefore

$$r_{\text{route}}(P) = \Pr(J \leq P),$$

where the probability averages over queries and a uniformly selected member of each query's true top  $K$ . This identity defines what to measure. Cluster count and average cluster size alone do not supply the distribution of  $J$ .

**B: follow bitplane branches.** Consider a four-level tree where a path specifies the first four bits. Keep every path with at most one disagreement with the query. It reaches the same five keys as the example above, so its recall is also 89.60%. If every allowed path is represented, the tree has  $1 + 2 + 3 + 4 + 5 = 15$  nodes, including the root and five leaves. Some nodes may have no documents and can be skipped.

This example counts a particular complete set of paths. The actual best-first algorithm has a different stopping rule. It chooses a pending node using its score bound, splits on a query coordinate, and may stop when the node budget is used. Its candidate set depends on the weights, occupied leaves, leaf threshold, current result threshold, and visit order. A budget of 15 nodes does not mean that it reaches the five prefix keys above. To estimate its recall, run that selection process on sampled data or derive its visited paths for a small explicit tree.

A random exploration probability changes which pending node is chosen. At a fixed budget it can recover a missed neighbor or spend work on an unhelpful path. Its effect must be checked with the same queries and budget. Complete traversal with valid bounds is the exact control.

**C: try keys near the query’s desired pattern.** With equal weights, opening every short key through a Hamming radius gives Equations (43) and (47). With unequal weights, keys must be ordered by the sum of the weights of their wrong bits. A large number of cheap mistakes can come before one expensive mistake. The weighted distribution in the next subsections replaces the simple mismatch count.

For full keys, an exact weighted order and a valid stopping rule give exact local binary results. Short keys omit part of the score, so stopping after enough postings have arrived can lose neighbors. It can also work well when the prefix strongly predicts the full ranking. The calculation must account for both possibilities.

Literal prefix relaxation is another concrete rule. For query key 0000, replacing the last constraint by a wildcard accepts both 0000 and 0001. A prefix trie can retrieve that parent range, or only the new sibling range if the first key was already read. Flipping the last bit instead visits only 0001. Relaxing arbitrary query-selected coordinates is not a single ordinary prefix-trie parent step. The candidate sets and data access costs must follow the chosen structure.

#### Example: Routing and lookup losses are conditional

Keep  $d = 16$  and condition on the nearest document having two errors. A router that demands the first two bits match retains it with probability  $\binom{14}{2}/\binom{16}{2} = 91/120$ . Given that it passed, demanding the next two bits match has probability  $\binom{12}{2}/\binom{14}{2} = 66/91$ . Their product is  $66/120 = 55\%$ .

Multiplying  $91/120$  by itself would give a different answer. Once the first two bits are known to match, the two errors must lie among the remaining fourteen positions. Conditioning changes the second calculation. The same issue arises when routing and lookup use overlapping bits.

## 6.10 A deterministic error bound for a short prefix

Split the full score into retained and omitted coordinates:

$$S = T_h + Z_h, \quad T_h = \sum_{i \in I_h} q_i b_i, \quad Z_h = \sum_{i \notin I_h} q_i b_i. \quad (49)$$

Because  $b_i \in \{-1, +1\}$ ,

$$|Z_h| \leq A_{\text{tail}} = \sum_{i \notin I_h} |q_i|. \quad (50)$$

Two documents can reverse their prefix order if the omitted contributions overcome their prefix score difference. A prefix gap greater than  $2A_{\text{tail}}$  is sufficient to preserve that pair’s order. This is a sufficient bound, not a typical-error estimate.

A more useful query-time certificate compares the best unseen prefix score plus  $A_{\text{tail}}$  with the worst fully scored retained result, as in Equation (31). It adapts to the actual query. If the bound is too loose, more keys must be visited. Returning a fixed candidate count without checking that condition remains approximate.

For float truncation, the analogous score error obeys Cauchy–Schwarz:

$$|q_{\text{tail}}^\top x_{\text{tail}}| \leq \|q_{\text{tail}}\|_2 \|x_{\text{tail}}\|_2.$$

Truncation and binary quantization are separate approximations. Matryoshka training can improve the usefulness of prefixes, but it does not set either bound to zero or supply a universal recall percentage.

### 6.11 Derive a weighted model before approximating it

Let  $E_i$  be one when document bit  $i$  disagrees with the query and zero otherwise. With  $a_i = |q_i|$ , the penalty is  $L = \sum_i a_i E_i$ . The score is  $\sum_i a_i - 2L$ , so ranking by largest score is the same as ranking by smallest  $L$ .

For query magnitudes  $[4, 3, 2, 1]$ , one error in the first coordinate costs 4. Errors in the last two cost  $2 + 1 = 3$ . The two-error document ranks higher. Counting wrong bits alone would reverse that order.

Under independent mismatch probabilities  $p_i$ , a particular error pattern  $e$  has probability

$$\Pr(E = e) = \prod_{i=1}^d p_i^{e_i} (1 - p_i)^{1-e_i}. \quad (51)$$

For a small dimension, list the  $2^d$  patterns, calculate each full penalty and prefix penalty, and add their probabilities into the corresponding score groups. That directly constructs the joint distribution needed for recall. It also works with dependent bits if we supply their actual joint pattern probabilities instead of the product above.

To handle a finite top  $K$ , group patterns by each distinct full penalty  $\ell$ . Let  $p_{\leq \ell} = \Pr(L \leq \ell)$  and  $p_{< \ell} = \Pr(L < \ell)$ . For a fixed candidate acceptance event  $A$ , the same reasoning as the equal-weight case gives

$$\mathbb{E}[\text{Recall}@K] = \frac{1}{K} \sum_{\ell} [G_K(p_{\leq \ell}) - G_K(p_{< \ell})] \Pr(A \mid L = \ell). \quad (52)$$

Each bracket counts expected top- $K$  rows at that penalty. The conditional probability says which of them pass. This is exact for independently sampled documents from the stated pattern distribution, with an ID tie rule independent of the sampled patterns. It does not require independence *between bits*. It does require the joint distribution to be known.

#### Example: Balanced bits can still be dependent

Make eight-bit documents by sampling four balanced bits and repeating them in the last four positions. Every individual bit is still zero or one half the time. However, only sixteen of the possible 256 codes can occur. The independent-bit model assigns probability to 240 impossible codes. A correction to the arithmetic cannot repair that assumption; the pattern probabilities must change. Section 7.4 checks this example explicitly.

At hundreds of dimensions, listing every pattern is impractical. We can sample from a stated model, use a suitable dynamic program, or make an approximation to the score distribution. The choice should follow the calculation we need. The small exact model remains useful for checking whether a faster calculation is answering the same question.

## 6.12 Random documents: what the independence assumption actually gives

The following distribution models first omit routing. When routing is added, retain the same global true-neighbor event or apply Equation (48); replacing it with a different local top- $K$  event changes the question. Assume nonzero score variance when computing a correlation.

First condition on a particular query  $q$ . Assume document signs are independent and equally likely to be  $-1$  or  $+1$ . Then

$$\mathbb{E}[S \mid q] = 0, \quad \text{Var}(S \mid q) = \sum_{i=1}^d q_i^2, \quad (53)$$

$$\mathbb{E}[T_h \mid q] = 0, \quad \text{Var}(T_h \mid q) = \sum_{i \in I_h} q_i^2, \quad (54)$$

$$\text{Cov}(T_h, S \mid q) = \text{Var}(T_h \mid q). \quad (55)$$

Hence the score correlation is exactly

$$\rho_q = \text{Corr}(T_h, S \mid q) = \sqrt{\frac{\sum_{i \in I_h} q_i^2}{\sum_{i=1}^d q_i^2}}. \quad (56)$$

High retained variance is not the same as a small worst-case tail bound. Many small omitted coordinates can have a modest sum of squares but a large sum of absolute values. A useful approximate prefix can therefore still lack a cheap exactness certificate.

The square root is important. Retaining a fraction  $v$  of score variance gives correlation  $\sqrt{v}$ , not recall  $v$  and not recall  $\sqrt{v}$ .

### Example: Two random-data cases can have very different prefix recall

With equal coordinate variances, a 24-coordinate prefix of a 256-coordinate query retains roughly 24/256 of expected score variance. If instead the first 24 coordinates are constructed to hold 80% of expected score variance, their score is much more strongly correlated with the full score. The controlled measurements in Section 7 test both cases. The second is a deliberately favorable distribution, not the definition of Matryoshka embeddings.

If query coordinates are random normals  $q_i \sim \mathcal{N}(0, \sigma_i^2)$ , we can choose a variance profile  $\sigma_i^2$ . The ratio of *expected* prefix and full variances is  $\sum_{i \in I_h} \sigma_i^2 / \sum_i \sigma_i^2$ . For a specific sampled query, use its actual Equation (56). The expectation of a ratio need not equal the ratio of expectations.

### 6.13 When document bits vary together

A covariance matrix records how coordinates vary together. Its diagonal entries are individual variances. An off-diagonal entry describes whether two coordinates tend to move together or in opposite directions. Independent, balanced signs have unit diagonal entries and zero off-diagonal entries.

For a general binary document vector with mean  $\mu$  and covariance matrix  $\Sigma$ , let  $q_h$  be  $q$  with omitted coordinates set to zero. Then

$$\rho_q = \frac{q_h^\top \Sigma q}{\sqrt{(q_h^\top \Sigma q_h)(q^\top \Sigma q)}}. \quad (57)$$

This includes dependencies between coordinates. Conditioning on a cluster or filter can change both  $\mu$  and  $\Sigma$ . A covariance measured globally is not automatically the conditional covariance inside the query's selected clusters.

A learned prefix may preserve ranking information through these dependencies even when its raw coordinate-energy share is small. Conversely, a prefix can have considerable energy and still fail to identify the extreme nearest neighbors. Estimating covariance also does not prove a normal distribution in the tails.

### 6.14 Changing the probability of a wrong bit

Let  $X_i = \mathbf{1}[b_i \neq t_i]$ . If the  $X_i$  are independent with the same disagreement probability  $p$ , and all relevant weights equal  $a$ , then a mismatch count over  $h$  coordinates satisfies

$$M_h = \sum_{i=1}^h X_i \sim \text{Binomial}(h, p), \quad \pi_h = aM_h. \quad (58)$$

A Hamming radius  $r$  visits

$$V_h(r) = \sum_{u=0}^r \binom{h}{u} \quad (59)$$

keys, and its expected candidate population is

$$N_* \Pr(M_h \leq r) = N_* \sum_{u=0}^r \binom{h}{u} p^u (1-p)^{h-u}. \quad (60)$$

Here  $N_*$  is the relevant reference pool size; it is  $N$  without filters or cluster restriction. Equation (60) is a candidate-count model, not yet recall.

If  $p = 1/2$ , every key has equal probability and the expected count reduces to  $N_* V_h(r)/2^h$ . Different  $p_i$  with equal weights give a Poisson-binomial count. Unequal weights give a weighted Bernoulli sum. Neither is described by one ordinary binomial with an average  $p$ .

### 6.15 Derive recall by asking which true neighbors pass

Write  $F_X(x) = \Pr(X \leq x)$  for a cumulative distribution function (CDF): the probability that  $X$  is at or below a threshold.

Let  $A_h$  be the prefix penalty and  $B_h$  the omitted-coordinate penalty. Full penalty is  $A_h + B_h$ . Under coordinate independence, the two sums are independent. Let  $u$  define the true full-score tail, and let  $v$  be the candidate prefix-penalty threshold. Then

$$r_{\text{population}}(u, v) = \frac{\Pr(A_h \leq v, A_h + B_h \leq u)}{\Pr(A_h + B_h \leq u)} = \frac{\int_{a \leq v} F_{B_h}(u - a) dF_{A_h}(a)}{F_{A_h + B_h}(u)}. \quad (61)$$

Read the numerator as a weighted sum: for each allowed prefix penalty, ask whether the remaining coordinates fit within the full-score threshold, multiply by the probability of that prefix penalty, and add.

This is the logic missing from a candidate-count-only calculation: a retrieved document counts toward recall only if it also belongs to the true full-score tail.

For equal weights and an integer full radius  $r_d$ , the formula becomes

$$\frac{\sum_{u=0}^{\min(h, r_h, r_d)} \binom{h}{u} p^u (1-p)^{h-u} F_{\text{Bin}(d-h, p)}(r_d - u)}{F_{\text{Bin}(d, p)}(r_d)}. \quad (62)$$

This is exact for the stated population events. Choosing those radii to approximate top  $K$  and a candidate budget is another step. Finite-sample ranks and score ties need care.

#### Example: Candidate fraction is not recall

Use all 64 six-bit codes as documents and an all-positive equal-weight query. The best seven codes have zero or one mistake. Keep only codes matching the first two query bits: 16 codes pass, so the candidate fraction is  $16/64 = 25\%$ . Five of the true seven pass: the ideal code and the four codes whose one error is in an omitted coordinate. Recall is  $5/7$ , about 71.4%, not 25%. Allowing one error in the two-bit prefix retrieves 48 codes and includes all seven. Both numbers follow by counting the same true-neighbor event.

### 6.16 Unequal query weights: count weighted subsets

For a fixed query and lookup coordinates  $I_h$ , define

$$V_q(v) = \# \left\{ S \subseteq I_h : \sum_{i \in S} |q_i| \leq v \right\}. \quad (63)$$

This counts every attempted key through threshold  $v$ , whether occupied or empty. Under independent uniform keys, expected candidates are  $N_* V_q(v)/2^h$ . With real key probabilities,

$$\mathbb{E}[F(v) \mid q] = N_* \sum_{k: \pi_q(k) \leq v} \Pr(\text{key} = k \mid q, \text{cluster}, \text{filter}). \quad (64)$$

For a clustered lookup, apply Equation (64) separately with pool size  $e_j$  and the conditional key distribution in cluster  $j$ , then sum the results.

An average bit-disagreement rate is insufficient to determine these joint probabilities. The empty-key count  $V_q$  and occupied candidate count  $F$  are separate inputs to the latency formula. A dense document distribution near the query key can make  $F$  large while  $V_q$  remains small; a sparse key space can do the reverse.

For small  $h$ , split weights in two, enumerate each half's subset sums, sort one list, and count pairs whose sum is at most  $v$ . This meet-in-the-middle calculation uses roughly  $O(2^{h/2})$  memory and  $O(h2^{h/2})$  sorting/search work. It is an analysis tool for counting keys, not the measured latency of an online hash enumerator.

For quantized nonnegative integer weights  $a_i$ , a dynamic program computes probability mass:

$$f_i(s) = (1 - p_i)f_{i-1}(s) + p_i f_{i-1}(s - a_i). \quad (65)$$

Replacing probabilities by counts and adding both predecessor counts computes key volumes. This is exact for the quantized-weight model. It does not silently become an exact calculation for unquantized query weights. If score quantization incurs a uniform score error at most  $\epsilon$ , a score gap greater than  $2\epsilon$  is sufficient to preserve a pair's order. Otherwise candidates near the boundary need exact scoring or a safe expanded bound.

## 6.17 The conditional-normal approximation

When a score adds many small independent contributions, a normal distribution can sometimes approximate its shape. Here we need a joint distribution: how the full score and prefix score vary together. This is an additional assumption, not a consequence of knowing their correlation. Subtract each score's mean and divide by its standard deviation. This standardizes them to  $(Z, Y)$  with correlation  $\rho$ . For a true fraction  $a = K/N_*$  and selected candidate fraction  $f$ , set

$$z_K = \Phi^{-1}(1 - a), \quad y_f = \Phi^{-1}(1 - f),$$

where  $\Phi$  and  $\phi$  are the standard normal CDF and density. Under the joint-normal assumption,  $Y$  conditional on  $Z = z$  has mean  $\rho z$  and variance  $1 - \rho^2$ . Its probability of exceeding the prefix threshold is therefore  $\Phi((\rho z - y_f)/\sqrt{1 - \rho^2})$ . Multiply by the full-score density  $\phi(z)$ , integrate over the true-neighbor tail, then divide by its probability  $a$ . The implied recall is

$$\hat{r} = \frac{1}{a} \int_{z_K}^{\infty} \phi(z) \Phi\left(\frac{\rho z - y_f}{\sqrt{1 - \rho^2}}\right) dz. \quad (66)$$

We average over the *entire* true tail. Substituting only  $z = z_K$  calculates a boundary conditional probability, not the integral. When  $\rho = 0$ , recall is  $f$ . In the limit of identical rankings,  $\rho = 1$ , it is  $\min(1, f/a)$ , subject to ties.

This formula is not valid simply because an embedding is called Matryoshka. A short weighted binary prefix has finite, discrete support. Its extreme-score distribution can differ greatly from a normal tail. At a billion documents, top- $K$  events may be far deeper in the tail than any small validation sample covers. Strong overall score correlation does not certify 99% recall.

The controlled study below fits score correlation on one query group and checks predicted recall on different queries. Even there, the real-data normal approximation misses by several percentage points. Such a model can generate hypotheses; it should not declare a real 99%-recall winner.

## 6.18 What can be concluded about the three methods

- **A:** exact local scanning has no local candidate-search loss; global loss comes from routing, candidate/reranker score changes, and the declared filtering task.
- **B:** exact bound-based traversal has the same binary result as A on the same selected pool. Finite budgets require query-level measurements or a distribution model for visited nodes and surviving true neighbors. A node count alone is not enough.
- **C:** full keys with complete weighted ordering are exact locally. Short keys can be accurate when their ranking preserves true neighbors and the candidate budget is sufficient. That possibility should be measured, not dismissed. Their enumeration and posting costs remain separate from the recall benefit.

A useful estimate needs a stated candidate rule and a distribution that fits the relevant neighbors. The checks below show both a case where the exact calculation works and a case where a plausible assumption gives the wrong answer.

## 7 What the existing measurements support

The purpose of these measurements is to check the reasoning, not to fit every equation until one method appears to win. The experiments have different scopes, so their numbers must not be merged into one unqualified performance claim.

### 7.1 The one-million-passage scaling study

The archived study used one flat cluster, a fixed 256-bit score, 100 binary candidates, leaf size 32, deterministic branching, cached query embeddings, and one CPU thread. There were 100 validation queries for selecting configurations and 200 held-out test queries, with three timing repeats. It covered pools from 1,000 to one million real MS MARCO passages.

| One-million-document test | Median latency | Binary recall | Fully scored rows |
|---------------------------|----------------|---------------|-------------------|
| Packed scan               | 25.14 ms       | 100%          | 1,000,000         |
| Branch, budget 32,768     | 244.98 ms      | 99.26%        | about 163,766     |
| Unlimited branch          | 1,068.84 ms    | 100%          | about 999,961     |

The branch reduced scoring at the finite budget, but did not win at matched target recall. The whole study found no median or p95 crossover among the tested 95% and 99% target settings. This conclusion belongs to that implementation, dataset, hardware and grid. It is not a theorem that every bitplane representation loses.

The original implementation uses dense active masks for every node. Its measured split-word counter omits leaf reads, which must be derived from the visited-node counts and cluster widths. The physical mask size at one million rows is  $15,625 \times 8 = 125,000$  bytes. A frontier with 20,000 such masks would contain 2.5 GB of masks alone. This example explains how query RAM can be much larger than the packed-code payload; it is not a claimed observed frontier count.

The source summary is `results/scaling-msmarco-1m/analysis/settings.csv`. Full run metadata and the validation protocol live beside it. The report’s verification script checks the numerical rows quoted here.

## 7.2 The FiQA study asked a broader pipeline question

The denser FiQA run used 57,638 documents and 648 queries, with 583 settings including random seeds. Its timings include routing, binary candidate search and float reranking, with embeddings cached. Its quality measures include float Recall@10 and nDCG@10, rather than the scaling study’s binary Recall@100.

| Method                 | Probes | Node budget | Median ms | Float Recall@10 |
|------------------------|--------|-------------|-----------|-----------------|
| Sign-route binary scan | 16     | –           | 0.7524    | 0.6784          |
| Sign-route branch      | 32     | 2,048       | 0.6256    | 0.7006          |
| Sign-route branch      | 64     | 2,048       | 0.6466    | 0.7062          |
| Float IVF reference    | 8      | –           | 0.1661    | 0.9127          |

These selected examples show that probing more clusters with limited work inside them can be a useful combined tradeoff. They do not isolate traversal efficiency: the selected pools differ. Within the same selected buckets, unlimited branching was slower than scanning. The float IVF row uses a different scorer and was a strong practical reference in that experiment.

Random exploration at probability 0.1 did not show a consistent improvement. Its purpose is to change visit order; it does not provide a mathematical guarantee of higher recall. The full FiQA report retains losing configurations, empty returns, and the relevant measurement scope. Its source is `results/fiqa-2026-09-09-dense/README.md` and the accompanying data.

## 7.3 A new controlled check of the 24-bit prefix

The small quality study used the same full weighted-binary score and row-ID tie rule for all methods. There was no outer routing:  $C = P = 1$ . Two synthetic datasets had 20,000 random binary documents. The real-data case used the first 100,000 rows of the cached Nomic/MS MARCO sample. All cases used  $d = 256$  and  $h = 24$ .

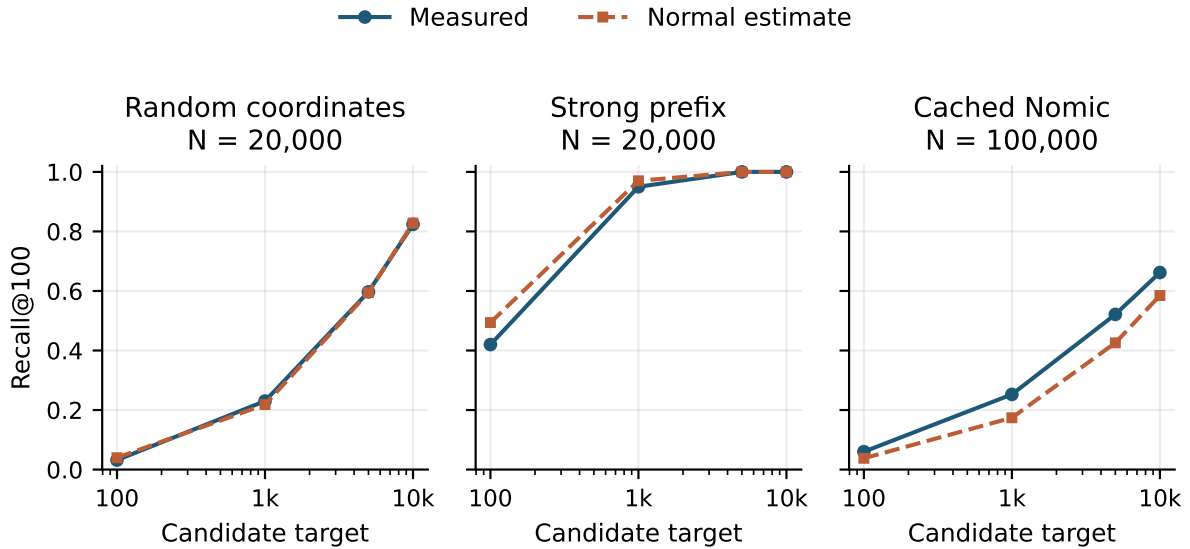
One synthetic query distribution gave all coordinates equal variance. The other multiplied the first 24 coordinates so that they held 80% of *expected* full-score variance. It was a controlled favorable case for a prefix, not a model of what Matryoshka always does.

For each case, 16 queries estimated a score correlation and 16 different queries checked the normal-model prediction. The native full scan agreed with an independent exact score sort on all 96 query references. The short-key candidate set included all documents in keys tied at the final prefix threshold, then used the exact full score to keep the top 100.

| Data, candidate target       | Measured recall | Normal prediction | Gap      | Median keys |
|------------------------------|-----------------|-------------------|----------|-------------|
| Random equal variance, 1,000 | 23.00%          | 21.84%            | −1.16 pp | 846,156     |
| Strong prefix, 1,000         | 95.00%          | 97.01%            | +2.01 pp | 846,156     |
| Cached Nomic, 1,000          | 25.25%          | 17.41%            | −7.84 pp | 6,976       |
| Cached Nomic, 5,000          | 52.13%          | 42.57%            | −9.56 pp | 35,504      |
| Cached Nomic, 10,000         | 66.19%          | 58.46%            | −7.73 pp | 78,370      |

“Gap” is prediction minus measurement in percentage points. Values are rounded from the per-query evidence. The identical synthetic key counts are expected: the strong-prefix case scales all first-24 query weights by the same factor, which changes neither their key order nor the prefix candidate sets. It changes their importance in the *full* score.

These results support two different conclusions. First, a short prefix is not inherently inaccurate: the constructed favorable case reaches 95% recall with 1,000 candidates. Second, a simple statistical prediction can still miss real recall by several points. An error of that size is unacceptable as a certificate of a 99% recall target.



**Figure 7:** Measured short-key recall versus the checked normal approximation. Each panel has its own explicitly stated document population.

The key counts were calculated from the actual query weights by meet-in-the-middle counting, including empty keys. The candidate set was constructed using an exhaustive prefix-score calculation for quality evaluation. **That computation is not an implemented hash-search latency benchmark.** Its wall time must not be plotted as the latency of Approach C.

In the same favorable synthetic case, native B at budget 512 recovered about 99.38% of the binary top 100 while fully scoring about 4,122 documents on average. In the real-data case, B reached about 58.06%, 86.56% and 99.56% at budgets 512, 2,048 and 8,192. These are quality and work observations for a flat pool with a 1,000-candidate heap. They do not establish a latency ranking between an implemented native B and a hypothetical C.

## 7.4 Check the new derivation against independently sampled corpora

The small arithmetic checker enumerates ordered corpora and sorts their documents directly. This includes duplicate codes, score ties, empty prefixes, full prefixes,  $K = 1$ , and  $K = N$ . All twenty settings agree with Equation (47) to numerical precision. The 16-bit worked example is then calculated from the formula rather than by enumerating every possible corpus.

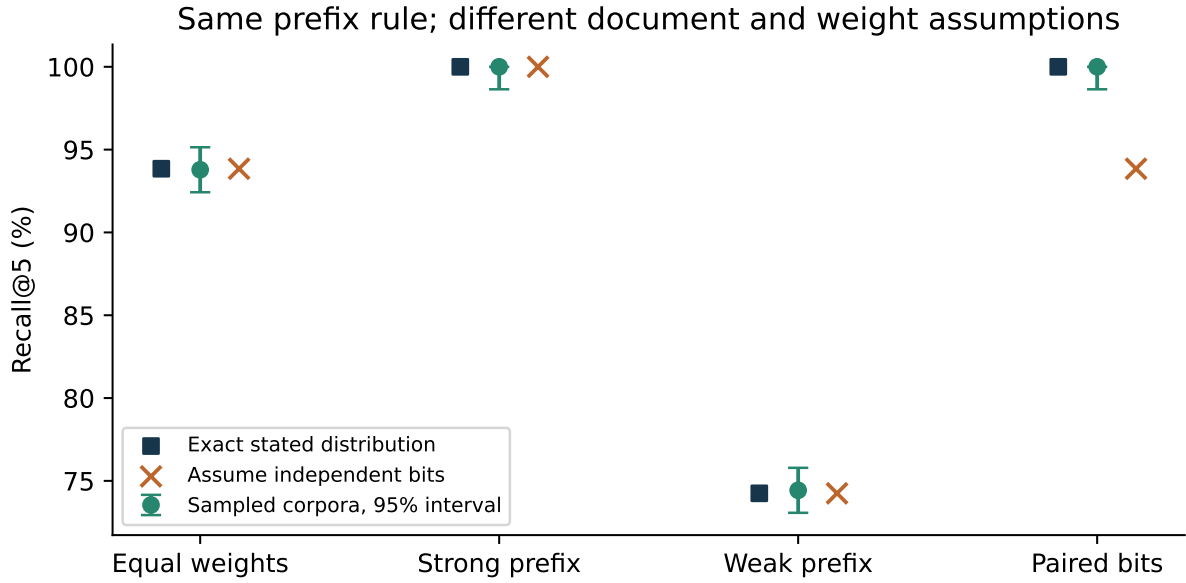
A second check uses eight-bit documents,  $N = 64$ ,  $K = 5$ , and the first three bits as lookup coordinates. A document passes when at most one of those three bits disagrees with the query. For each distribution, the script calculates the exact expected recall by grouping all possible patterns by full penalty, then samples 10,000 independent corpora and checks the returned top five directly. The query has positive coordinates with the following magnitudes:

- Equal weights:  $[1, 1, 1, 1, 1, 1, 1, 1]$ ; all bits independent and balanced.
- Strong prefix:  $[4, 3, 2, 1, 1, 1, 1, 1]$ ; the same independent document distribution.
- Weak prefix:  $[1, 1, 1, 4, 3, 2, 2, 2]$ ; the same independent document distribution.
- Paired bits: all query magnitudes one, with the last four document bits repeating the first four. Only sixteen patterns can occur.

These are deliberately simple assumptions. They let us change one cause at a time without calling the resulting distribution a model of a particular embedding system.

| Case          | Exact stated model | Sampled mean | Assume independent bits |
|---------------|--------------------|--------------|-------------------------|
| Equal weights | 93.847%            | 93.784%      | 93.847%                 |
| Strong prefix | 99.999%            | 100.000%     | 99.999%                 |
| Weak prefix   | 74.248%            | 74.430%      | 74.248%                 |
| Paired bits   | $\approx 100\%$    | 100.000%     | 93.847%                 |

The candidate rule is identical in every row. It returns an expected 32 of the 64 documents, but its recall changes with the score weights and document distribution. The last row isolates the failed assumption: individual bits remain balanced, yet assuming they are independent underestimates recall by about 6.15 percentage points.



**Figure 8:** Exact calculations and independently sampled corpora agree for the stated distributions. The paired-bit row shows the error from using the wrong distribution.

Each sampled corpus contributes one recall value in  $[0, 1]$ . A conservative uncertainty bound for their mean follows from

$$\Pr(|\bar{r} - \mathbb{E}[r]| \geq \epsilon) \leq 2e^{-2n\epsilon^2}.$$

Setting the right side to 0.05 gives  $\epsilon = \sqrt{\log(40)/(2n)}$ , about 1.36 percentage points at  $n = 10,000$ . The plotted intervals use this bound and are clipped to  $[0, 1]$ . This remains meaningful when no misses were observed. A zero sample variance by itself would produce a misleading zero-width normal interval near 100%. The bound assumes independent sampled corpora; it does not turn ten thousand simulations into evidence about real embeddings.

For the first twelve sampled corpora in each case, the check also calls the current native scan and bitplane search. All 48 full scans and all 48 unlimited branch searches match the independently sorted top five. With a 16-node budget and leaf size four, mean branch recalls are 80.00%, 98.33%, 100.00%, and 85.00% in the table’s order. At exploration probability 0.1 and the same budget, they are 75.00%, 98.33%, 100.00%, and 85.00%. These small examples show that the budget and weights affect which neighbors survive. They are not estimates of performance on a large dataset or evidence that random exploration always hurts.

The script separately reaches the same fixed prefix-radius candidate set through key grouping and through the union of bitplane leaves. All 48 candidate sets agree exactly. This checks the shared-set argument; it does not claim that budgeted best-first traversal visits those same leaves. No latency ranking is inferred from these quality checks.

## 7.5 Put an uncertainty interval around the saved prediction errors

The earlier study already showed that the normal approximation could miss real recall. To see how variable that gap is across queries, resample its sixteen check queries with replacement 10,000 times. For each resample, average the paired difference  $\hat{r}(q) - r(q)$ . The middle 95% of those averages gives the bootstrap interval below. A percentage point is an absolute difference: predicting 30% when the observation is 25% is an error of +5 points.

| Case          | Candidate target | Mean error (points) | 95% interval    |
|---------------|------------------|---------------------|-----------------|
| Random signs  | 1,000            | -1.16               | [-3.72, 1.41]   |
| Strong prefix | 1,000            | +2.01               | [0.08, 4.14]    |
| Cached Nomic  | 1,000            | -7.84               | [-14.47, -1.54] |
| Cached Nomic  | 5,000            | -9.56               | [-18.25, -0.81] |
| Cached Nomic  | 10,000           | -7.73               | [-16.17, 0.64]  |

This is a reanalysis of previously examined queries, not a fresh test set. The fitted score correlation is held fixed, so the intervals do not include uncertainty from estimating that correlation. They also do not cover corpus changes, model changes, or selection among many configurations. When every checked query gives the same outcome, bootstrap variation can vanish; that is not proof that future queries have no variation.

A proposed requirement of at most two points of mean error fails on every cached-Nomic setting shown here. Even the random-sign row, whose observed mean error is small, has an interval that extends beyond that tolerance. We should therefore retain this normal model as an explanation of a possible mechanism, rather than use it to certify real 99% recall.

Improving the prediction requires new evidence about the relevant score distribution. For example, estimate the joint full and prefix penalties on calibration queries, freeze the candidate policy, and check it on separate queries. If an empirical distribution is used, its resolution in the nearest-score tail must be reported. A small sample can miss rare but important patterns; moving from 100,000 to a billion documents makes that issue more severe. The current extension does not claim a newly validated real-data predictor.

## 7.6 What was not validated

The study does not validate:

- real recall at a billion documents;
- a routing-recall model across arbitrary cluster counts;
- a normal tail for every query or a fixed percentage of Matryoshka prefix information;
- a concrete hash enumerator’s timing, peak queue size, or SIMD speed;
- float-vector recall, human relevance, or final answer accuracy from binary recall alone.

Float-vector top-100 results and binary-score top-100 results answer different questions. This report keeps their comparisons separate. A method that searches binary scores exactly can still miss neighbors under the original float-vector score.

## 7.7 How to extend the checks fairly

Freeze the representation, query set, filtering task and reference score first. Tune on one query set; check selected configurations on a different one. For every method, retain failures and empty returns, and record counts as well as times. For C, implement the actual enumerator before claiming its measured latency. For B, record split words, leaf words, documents scored and peak frontier memory separately.

At each target recall, compare measured latency quantiles using configurations that actually reach that target. Use the same query population and report the uncertainty of quality estimates. Query-level resampling is appropriate; counting repeated timings as independent quality samples is not. A few measured cases are useful evidence about mechanisms, not a complete characterization of every data distribution.

## 8 From the formulas to an implemented comparison

The earlier studies exposed two gaps. The branch counter missed the work of reading leaf masks, and the short-key recall experiment did not measure a native key index. The next study implements all three methods with the same score, then checks the pieces before comparing complete requests. This section reports that study separately from the historical results.

### 8.1 What is shared, and what changes

A, B and C use the same C++ lookup-table scorer and the same top- $K$  heap. Each query remains floating point; each document stores one sign bit per coordinate. The final score is the full weighted binary score from Section 1, with smaller document IDs first when scores tie. There is no float reranker in this comparison. Thus recall means recovery of the exact binary-score top 100, not relevance to a person reading the documents.

The Python driver supplies cached query vectors and selected cluster IDs. A fresh process builds one configuration, releases its input mappings, warms up a query, then measures routing and search together. The native search also records its own elapsed time. Measuring the outer call includes the Python/native boundary; reporting only the native timer would omit it.

The machine is an Apple M5 Max with 128 GiB of physical memory. Each worker uses one CPU thread and handles one query at a time. The comparison budget is 32 GB, in decimal bytes. The machine’s physical capacity and the experiment’s memory constraint are different numbers. Power configuration is recorded before and after each worker. Equal settings do not establish fixed CPU frequency or a completely idle machine, so timings are repeated in separate processes.

### 8.2 The implemented key directory

The native C build sorts rows by key inside each cluster and stores their full codes in that same order. Its existing document IDs are the postings. There is no extra  $N$ -element array of posting references. This is the layout alternative described after Equation (30).

The directory is a C++ `unordered_map`. It uses separate chaining: occupied entries are allocated as nodes and reached through a bucket-pointer array. Equation (29) describes a different, open-addressed layout and must not be used as its measured allocation. Let  $L_j$  be the number of hash

buckets,  $b_{\text{node}}$  the allocated node size including fields and alignment, and  $b_{\text{ptr}}$  a bucket-pointer size. Then the implemented allocation is

$$M_{C,\text{allocated}} = M_{\text{codes},\text{capacity}} + M_{\text{IDs},\text{capacity}} + \sum_j (U_j b_{\text{node}} + L_j b_{\text{ptr}}) + M_{\text{containers}} + M_{\text{allocator}}. \quad (67)$$

The last two terms cover container objects and allocator bookkeeping. They depend on the standard library and build. The logical directory payload is easier to check exactly: each occupied entry holds a key and a start/count pair,

$$M_{\text{directory},\text{payload}} = \sum_j U_j (b_k + 2b_o).$$

In this build those fields occupy 4, 8 and 8 bytes respectively. Their 20-byte sum is a payload count, not a claim that an allocated hash node occupies only 20 bytes.

B retains packed document rows and a second, transposed bitplane representation. A is built without bitplanes. Giving A the unused plane allocation would distort the memory comparison. All three retain the IDs needed to return original document rows.

### 8.3 The implemented key search uses one stream

The implementation has the same free key coordinates in every selected cluster. It enumerates one weighted sequence of keys and looks up each key in every selected cluster. This differs from the per-cluster target example earlier in the report.

**Listing 13:** The native key-search policy used in the measured study.

```
SearchKeys(query, selected_clusters, K, candidate_target, lookup_limit):
    build the shared full-score lookup table
    order the key coordinates by increasing absolute query weight
    queue = [the ideal key, with mismatch penalty zero]
    best = empty top-K heap
    while queue is not empty:
        if best is full and the best remaining score bound cannot improve it:
            stop
        state = remove the lowest-penalty flip set
        key = ideal key XOR state.flip_mask
        for cluster in selected_clusters:
            if the lookup limit is reached: return best
            count one lookup, including an empty lookup
            for document in directory[cluster][key]:
                score all document coordinates; offer the score and ID to best
            if a nonzero candidate target is reached: return best
        add the next flip sets to the queue
    return best in score/ID order
```

The score bound is the full ideal score minus twice the key’s mismatch penalty. Coordinates outside the key are allowed their best possible contribution. This remains a safe upper bound when routing fixes some of those coordinates, although it may be loose. The implementation does not yet tighten it separately for each cluster.

The candidate target is checked after finishing a key in all selected clusters. A lookup limit can stop in the middle of that cluster loop. Both limits are approximations; zero disables a limit. The exact score bound remains available in either case. Equal-score candidates require the shared ID rule, so the bound includes conservative floating-point slack.

There is one enumeration queue, not  $P$  separate queues. Charge its preparation and generated states once; charge each cluster lookup separately. Every returned candidate still receives all  $G$  score-table terms. The tail-only and full-key score reuse discussed earlier are possible alternatives, but they are not credited to the measured implementation.

## 8.4 Check the work before fitting milliseconds

Logical code, ID, plane and key-directory bytes are checked against separate Python formulas. Native array capacity is reported separately because a vector may reserve more entries than it currently holds. The tests also compare returned IDs against an independent full-score sort. The reference calculation starts from document signs, not from the native packed scorer.

B reports split-word visits and leaf-word visits separately. C reports generated keys, directory lookups including misses, and fully scored documents. Both report their largest tracked queue or mask allocation. These counts explain a timing result; a low document-score count alone does not establish that the whole query performed little work.

### Example: Two cuts do not shorten the physical bitmap

Use every possible eight-bit document once, so  $N = 256$  and  $W = \lceil 256/64 \rceil = 4$  words. The first three query coordinates have magnitude one; the remaining five have magnitude 0.0001. Request four results and stop splitting when 64 documents remain.

Two matching cuts leave  $256/2^2 = 64$  documents. The search visits the root, its preferred child, and the leaf: three nodes. It reads  $2 \times 4 = 8$  split words and another four words to enumerate the leaf. It then fully scores 64 documents. At the second split, one queued alternative, the parent and both new children occupy four masks, or  $4 \times 4 \times 8 = 128$  bytes. The native measurement matches each count and returns the independently calculated top four. With a one-node budget it returns no candidates: that neighboring truncated case checks that the test can observe missing results. This example establishes the stated path, not a universal bound of two cuts for arbitrary query weights.

## 8.5 Fit elapsed time, then try different sizes

Instruction counts cannot be multiplied by a universal “nanoseconds per operation.” A lookup that hits a nearby cache line and a lookup that fetches a new memory page have different costs. The first timing model therefore uses a small set of measured work features with nonnegative coefficients. Its search-time inputs are:

- A: a fixed term and the number of full score-table terms.
- B: the same terms, split plus leaf word visits, and visited nodes.
- C: the same score terms, directory lookups, and a queue-work estimate.

The coefficients combine effects of arithmetic, memory access and result selection. They are not individual CPU-instruction timings. Measured routing time is added per query before taking a

median; summing the medians of separate stages would not give the median complete request.

There were 117 calibration configurations on random sign data with one cluster. An initial fit minimized squared errors in milliseconds. Long queries dominated that objective and some short branch queries had large relative errors. Refitting the same calibration data using squared percentage errors improved the diagnostic comparison. The original fit remains saved.

The revised coefficients were then frozen and checked on 52 configurations at different sizes and a new seed. All four scan settings and all 24 key settings had median prediction error within 20%. For B, 21 of 24 settings met that criterion; its worst error was 27.2%. The failures remain in the tables. These coefficients describe a limited range of working-set sizes, not an established billion-document latency model.

This model also uses *observed* work counts. Predicting how many branches or empty keys a new query will visit is a different problem, governed by the document and query distribution. A fit that converts measured work into milliseconds does not validate a forecast of recall or candidate count. The later tuning comparisons use measured latency, rather than allowing a poorly transferred model to choose a winner.

## 8.6 Give the baseline enough parameter choices

The real-data study reuses 256-bit Nomic/MS MARCO document codes at  $N = 10,000, 100,000$  and  $1,000,000$ . All methods get the same candidate router families and cluster layouts; each may select its own fastest tested configuration at a target recall.

A coarse 100,000-document run initially suggested a B advantage near 95% recall. The scan baseline’s next available probe setting jumped from 92.76% recall to a full scan. Adding intermediate probe counts removed that apparent advantage. This was a gap in the experiment, not evidence that B’s traversal was faster.

The later probe choices use the actual ranks of the correct neighbors’ clusters. Let  $r_{qi}$  be the rank of the cluster containing correct neighbor  $i$  for tuning query  $q$ . For  $Q$  queries and top  $K$ , the exact local-scan recall after probing  $P$  clusters is

$$\hat{R}_{\text{route}}(P) = \frac{1}{QK} \sum_{q=1}^Q \sum_{i=1}^K \mathbf{1}[r_{qi} \leq P]. \quad (68)$$

Sorting these  $QK$  ranks gives the smallest empirical cutoff for any requested target. The actual router checks the proposed cutoff; tied cluster scores are handled explicitly. This closes a known probe-grid gap without claiming that recall is determined by  $P/C$ .

## 8.7 Freeze settings, then use different queries

The refined runs were combined into 6,821 distinct settings. Equivalent configurations pool their timing observations. Random exploration uses a fixed set of seeds, averaged per query; the fastest lucky seed is not selected as the method’s result. The chosen parameters were frozen before evaluating 200 query IDs excluded from the original 300-query collection.

Two selection rules were retained. The *mean* rule picks the fastest measured setting whose tuning recall reaches the target. The *conservative* rule requires the lower bootstrap percentile of tuning recall to reach it. This second rule leaves more margin, but it is still a rule fitted to a finite sample, not a guarantee about all future queries.

There are 72 distinct frozen settings. Their predeclared seeds, three process blocks and three timing repeats produced 258 workers and 154,800 query observations. All workers completed and kept the same recorded power configuration. The largest process lifetime peak was 231,129,088 bytes, below the 32 GB constraint for these measured runs. It includes construction and runtime memory, so it is more conservative than the logical index payload alone.

Only 7 of 36 mean-selected target points reached their requested recall on the new queries. All 36 conservative points did. Re-encoding three old query texts with the fresh-query recipe exactly reproduced their cached vectors; no parameters were changed in response to the misses. A tuning average that just reaches 99% is not a promise of 99% on another finite query sample.

| <i>N</i>  | Target | Median milliseconds |        |         | Achieved recall (%) |       |       |
|-----------|--------|---------------------|--------|---------|---------------------|-------|-------|
|           |        | A                   | B      | C       | A                   | B     | C     |
| 10,000    | 80%    | 0.0979              | 0.1052 | 0.1129  | 84.91               | 84.91 | 84.91 |
| 10,000    | 90%    | 0.1339              | 0.1474 | 0.1560  | 91.79               | 91.79 | 91.79 |
| 10,000    | 95%    | 0.1683              | 0.1818 | 0.1968  | 95.84               | 95.84 | 95.84 |
| 10,000    | 99%    | 0.2330              | 0.2542 | 0.2812  | 99.27               | 99.27 | 99.27 |
| 100,000   | 80%    | 0.2345              | 0.2454 | 0.2698  | 80.55               | 80.55 | 80.55 |
| 100,000   | 90%    | 0.5034              | 0.5293 | 0.5918  | 90.98               | 91.29 | 91.29 |
| 100,000   | 95%    | 0.7527              | 0.7976 | 0.9308  | 95.47               | 95.47 | 95.57 |
| 100,000   | 99%    | 1.6219              | 1.6779 | 1.8109  | 99.44               | 99.44 | 99.44 |
| 1,000,000 | 80%    | 0.6749              | 0.6985 | 0.7506  | 82.53               | 82.53 | 82.53 |
| 1,000,000 | 90%    | 1.6341              | 1.6856 | 1.8802  | 91.73               | 91.73 | 91.73 |
| 1,000,000 | 95%    | 3.3600              | 3.4295 | 3.7055  | 95.47               | 95.47 | 95.47 |
| 1,000,000 | 99%    | 9.5796              | 9.9645 | 11.0127 | 99.25               | 99.25 | 99.25 |

The table reports the conservative choices. Each method can choose different cluster counts, probe counts and local settings. “Target” is the common minimum requirement; their achieved recalls need not be numerically identical. Exact settings and all mean-rule target misses are saved alongside the measurements.

Uncertainty is calculated by resampling queries and whole process blocks, using the same draws for the paired methods. Timing repetitions are not treated as independent recall observations. A supported alternative speedup requires both settings to meet the target and memory/power checks, and the paired 95% interval for  $t_A/t_{\text{alternative}}$  to lie above one. None of the 48 evaluated comparisons meets that rule. These intervals describe individual comparisons; they are not simultaneous guarantees across every setting.

The measured conclusion is therefore narrow but useful: for this cached binary score, data, single-thread implementation and tested parameter space, the independent evaluation did not establish a B or C speedup over the tuned scan baseline. It does not exclude different query weight distributions, other data layouts, hardware kernels or larger corpora.

## 8.8 How to reproduce this chain

Start with `experiments/README.md`. The small configuration checks components without a dataset download. The isolated calibration and confirmation configurations check the timing model. The frozen real selection lives in `results/frozen-real-2026-09-10/`; its independent measurements and uncertainty are in `results/independent-evaluation-2026-09-10/`. Each

run stores its configuration, source and native hashes, environment, per-query observations and process status. Raw-case ZIPs are verified against every original file before archival.

The implementation follows a short path through the repository:

| Paper topic                        | Code to inspect                                                                         |
|------------------------------------|-----------------------------------------------------------------------------------------|
| Shared binary score                | <code>cpp/score.hpp</code>                                                              |
| Full scan and bitplane traversal   | <code>cpp/index.cpp</code>                                                              |
| Weighted key enumeration           | <code>cpp/key_index.cpp</code>                                                          |
| One complete measured query        | <code>experiments/worker.py</code>                                                      |
| Independent storage/work formulas  | <code>experiments/models.py</code>                                                      |
| Pooled tuning choices              | <code>experiments/choices.py</code>                                                     |
| Frozen-query replay and comparison | <code>experiments/evaluate.py</code> ,<br><code>experiments/evaluation_report.py</code> |

## 9 A distribution where the important coordinates are known

Real embeddings did not establish an advantage for B or C in the independent comparison. To understand the mechanism, we can construct a distribution in which the score tells us exactly which coordinates matter most. This is an experiment about the algorithms, not a claim that a trained Matryoshka embedding has this distribution.

### 9.1 State the assumptions before calculating recall

Let document signs be independent and balanced: each coordinate is independently  $+1$  or  $-1$  with probability  $1/2$ . A query has  $m$  strong coordinates of magnitude  $a$ , and  $d - m$  weak coordinates of magnitude  $\epsilon > 0$ . Its signs are independently random. Scaling the whole query to unit length does not change any ranking or the following inequality:

$$(d - m)\epsilon < a. \quad (69)$$

A single strong mismatch loses  $2a$  score. Even changing every weak coordinate from its best to its worst sign loses only  $2(d - m)\epsilon$ . Therefore any document matching all strong coordinates outranks every document with a strong mismatch. More generally, fewer strong mismatches always outrank more strong mismatches; the weak coordinates decide the order inside one mismatch count.

The experiment sets  $d = 256$ ,  $m = 13$ ,  $a = 1$  and  $\epsilon = 0.0001$ . The weak total is  $0.0243$ , so Equation (69) holds. One setup puts the strong coordinates first. The other chooses a different set of 13 coordinates for each query. Neither changes the document distribution. All 40 generated queries are retained, with the first 20 used for tuning and the remaining 20 for checking frozen choices.

### 9.2 Recall from opening only the ideal strong key

Let  $Z$  count documents matching all  $m$  strong signs. Under the stated independent distribution,

$$p = 2^{-m}, \quad Z \sim \text{Binomial}(N, p), \quad \mathbb{E}[Z] = N2^{-m}.$$

Suppose a method opens exactly that key, scores every document in it, and returns at most  $K$ . Every one of those documents outranks all documents outside the key. Its recall on this corpus

is therefore exactly

$$R_{\text{ideal}} = \frac{\min(Z, K)}{K}. \quad (70)$$

For the expected recall over random corpora, use the fact that  $\min(Z, K)$  counts one for every threshold  $i \in \{1, \dots, K\}$  that  $Z$  reaches. Taking expectations gives

$$\mathbb{E}[R_{\text{ideal}}] = \frac{1}{K} \sum_{i=1}^K \Pr(Z \geq i). \quad (71)$$

This is a finite binomial calculation. It does not replace  $Z$  by its average and then assume that every query has that many matching documents.

**Example: Why a million documents can make a short key more useful**

At  $m = 13$ , a 100,000-document corpus has about 12.2 ideal matches on average. Opening only that key cannot usually return the top 100. At one million documents, the average is about 122.1, so one key often contains enough results.

In the independent fixed-coordinate sample, 19 queries had at least 100 ideal matches and one had 99. Equation (70) predicts mean recall  $(19 \times 100 + 99)/(20 \times 100) = 99.95\%$ . The scan that opened only that cluster measured exactly 99.95%. No fitted embedding-accuracy percentage was needed.

If the strong coordinates move between queries, a fixed first- $m$  key may contain only a few of them. Equation (70) then does not describe that key's recall. The assumption is about the coordinates actually indexed, not simply the number of indexed bits.

### 9.3 How many keys must exact lookup try?

For a key containing exactly the  $m$  strong coordinates, equal strong magnitudes make all keys with  $r$  mismatches share a prefix penalty. Let  $Z_r$  be the number of documents whose strong key has exactly  $r$  mismatches. For a fixed query,

$$\Pr(\text{one document has } r \text{ strong mismatches}) = \binom{m}{r} 2^{-m}.$$

The vector of counts follows a multinomial distribution; the counts at different radii are not independent. Let  $r_*$  be the first radius with at least  $K$  documents in total:

$$r_* = \min \left\{ r : \sum_{i=0}^r Z_i \geq K \right\}.$$

The measured exact key search finishes the tied key layer at that radius. Under Equation (69), it can then reject all larger radii. Its work is

$$H = \sum_{i=0}^{r_*} \binom{m}{i}, \quad F = \sum_{i=0}^{r_*} Z_i. \quad (72)$$

$H$  includes empty keys.  $F$  counts full document scores in the present implementation. For  $r_* = 1$  and  $m = 13$ , that means 14 key attempts, not one attempt and not  $2^{13}$  attempts. A smaller lookup limit invalidates the exact-stop conclusion. A candidate quota can also stop before the layer is complete, so its recall must be measured using the actual stopping rule.

The probability that radius  $r$  contains enough documents is another binomial tail:

$$\Pr(r_* \leq r) = \Pr\left(\text{Binomial}\left(N, 2^{-m} \sum_{i=0}^r \binom{m}{i}\right) \geq K\right).$$

This supplies a distribution over work, rather than declaring that the expected occupancy is an exact count for every query. No independence assumption between the different radii is needed.

#### 9.4 Choose how far bitplanes should split

B orders the coordinates using the actual query magnitudes, so it can follow the strong signs even when their positions change. Suppose at least  $K$  documents match all  $m$  strong signs. After following  $j \leq m$  preferred splits, those  $K$  correct results are still present. Score the remaining set and the full-score bound can reject the alternatives that missed a strong bit.

With  $W = \lceil N/w \rceil$  physical words per mask, the one-path approximation is

$$T_B(j) \approx T_{\text{prepare}} + jWc_{\text{split}} + Wc_{\text{leaf}} + N2^{-j}c_{\text{score}} + T_{\text{queue}}(j). \quad (73)$$

Both split and leaf loops keep the original width  $W$ . For an actual corpus, replace  $N2^{-j}$  by the observed surviving count. If fewer than  $K$  documents match all strong coordinates, more branches may be required and this single-path expression is insufficient; those queries stay in the study.

One more cut saves about  $N2^{-(j+1)}$  document scores. It helps only when those saved scores cost more than another full-width split and the added queue work:

$$N2^{-(j+1)}c_{\text{score}} > Wc_{\text{split}} + T_{\text{queue}}(j+1) - T_{\text{queue}}(j). \quad (74)$$

Ignoring the smaller queue term gives a first estimate from the derivative:

$$\frac{dT_B}{dj} = Wc_{\text{split}} - N \ln(2) 2^{-j}c_{\text{score}} = 0, \quad j_c = \log_2 \left( \frac{Nc_{\text{score}} \ln 2}{Wc_{\text{split}}} \right).$$

The second derivative of the score term is positive. Check the neighboring integers and boundaries 0 and  $m$ , then check real leaf sizes near  $N2^{-j}$ . This optimizes one stated model. It does not optimize arbitrary clustering, discrete query paths and hardware behavior with one derivative.

When  $N$  is large,  $N/W$  is about the fixed word width  $w$ . Thus  $j_c$  need not grow with  $N$ . A fixed leaf size can be a poor scaling choice: it forces extra cuts as the corpus grows, even when another full-width mask pass costs more than the document scores it removes. Both global dense-bitplane traversal and fixed-dimension scan still have at least linear work in  $N$  in this model. B can improve the constant; the first full-width pass does not become a logarithmic document lookup.

For  $j \geq 1$  with nonempty alternatives, the last split can hold  $j+2$  full masks: queued alternatives, the parent and both children. The corresponding mask payload is

$$M_{\text{masks,peak}} = (j+2)Wb_w.$$

With no split, only the root mask is needed. Queue metadata, score tables, outputs and the persistent index are additional memory.

## 9.5 What happened when the strong coordinates stayed fixed

All methods could use direct-key routing as well as the centroid and sign-prefix scans. There were 704 tuning configurations and 27 independent evaluation workers. The fastest tested key configuration used one physical cluster and a 13-bit key. The scan baseline chose direct routing into 8,192 clusters, which uses the same 13 strong signs.

At one million documents, C measured 0.0127 ms median latency and 100% binary recall. A measured 0.0145 ms and 99.95% recall. Their paired speedup estimate was 1.145, with a 95% interval of about 1.095 to 1.196. Both met every requested target through 99%. At 100,000 documents and the 99% target, both reached 100% recall: C measured 0.0151 ms and A 0.0186 ms, a paired estimate of 1.227 with interval about 1.144 to 1.364.

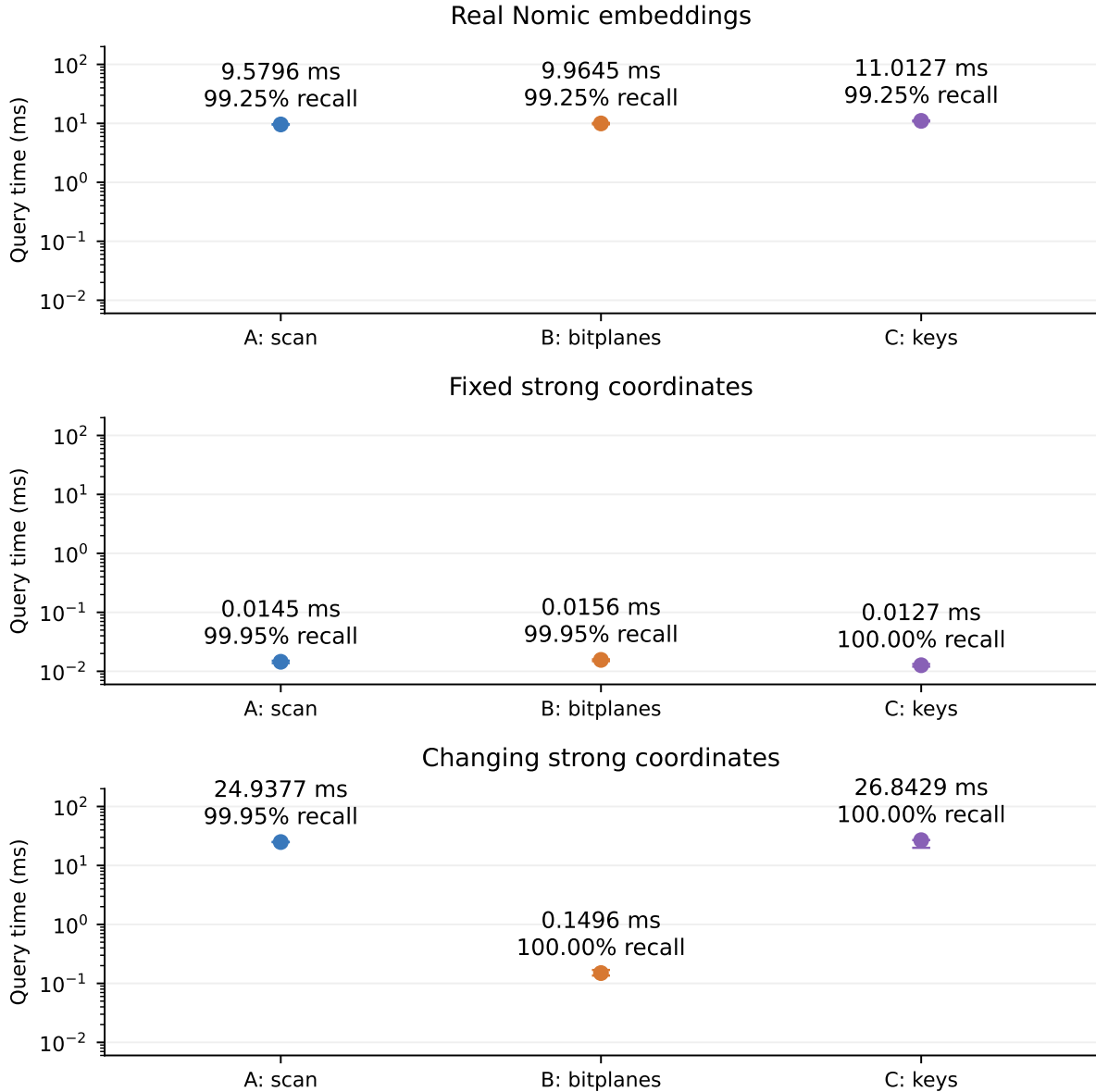
The difference is a few microseconds, and the methods largely overlap in candidate selection. C's key enumeration and scoring share one native call. A first ranks cluster keys through a native call, then scans their rows through another. The measurements establish an implementation advantage for this integrated lookup, not a theorem that key lookup dominates a scan baseline that incorporates the same candidate-selection rule. Larger cluster and key grids may also change the fastest tested setting.

## 9.6 What changed when the strong coordinates moved

The second setup uses the same document signs, but chooses each query's strong coordinates uniformly without replacement. Fixed-prefix keys cannot follow those changing positions. B can, because its coordinate order comes from the query itself.

This run measured 1,001 tuning settings and 60 independent evaluation workers. At one million documents and the 99% requirement, the frozen B configuration used one cluster, unlimited traversal and leaf size 128. It reached 100% binary recall at 0.1496 ms median latency. The scan configuration used 256 centroid clusters and 255 probes, reaching 99.95% at 24.9377 ms. Their paired speedup estimate was about 166.7, with a 95% interval of 147.6 to 183.9. C reached 100% at 26.8429 ms using one cluster and a 12-bit key.

At 100,000 documents, the 99% target comparison reached 100% recall for all three: A measured 2.5438 ms, B 0.1375 ms, and C 2.7170 ms. B's paired speedup over A was about 18.5, with a 95% interval of 18.1 to 19.0. These are selected-settings comparisons within the declared grid, with query and process-block uncertainty. They are not proofs of a global optimum over every cluster count, leaf size, routing algorithm or implementation.



**Figure 9:** Independent measurements at one million documents, a 99% binary-recall requirement and a 32 GB memory limit. Labels show median time and achieved recall. The vertical axis is logarithmic: adjacent major ticks differ by a factor of ten. The real-data evaluation has 200 queries; each constructed case has 20.

## 9.7 Compare predicted work with the saved native counters

The count checker uses document signs recorded during preparation, independently of native index traversal. It checks the B’s single-path formula only when at least  $K$  documents match all strong coordinates and a leaf is reached before splitting weak coordinates. For C, it checks the single-key bound case and Equation (72) when the key contains all strong coordinates. Empty keys are counted, including keys in the final tied layer.

Across the completed tuning and evaluation runs, 447 applicable per-query predictions matched the native counters exactly. This includes split words, leaf words, full scores, key attempts and the tracked B mask peak where applicable. Other queries and clustered configurations remain in the benchmark; the checker reports them as outside these specific prediction conditions. It does

not silently treat them as successful predictions.

There are two different results here. The component and conditional-work checks support the stated formulas. The timed experiment establishes how the implemented methods performed on these data. Neither alone establishes a universal latency forecast. In particular, if a new machine changes the relative cost of a bitmap pass and a full score, Equation (74) can favor a different leaf size.

The complete settings and raw evidence live under `results/controlled-fixed-2026-09-10/` and `results/controlled-adaptive-2026-09-10/`. The small runnable example is `examples/small_search.py`; it prints one query, its stored document bits, exact results and each method's work. `experiments/check_controlled.py` connects the conditional formulas to the recorded counters.

## 10 Check the complete cost, not just the counters

Correct work counts are necessary, but they do not make a latency prediction exact. Two loops can visit the same number of rows while reading very different memory locations. A small candidate set reused repeatedly can stay in cache; different candidates may require fresh memory reads. The checks below separate those effects and then return to complete native calls.

### 10.1 Measure buffers at the sizes the formulas describe

The standalone profile uses the shared C++ score table and top- $K$  code. Its path operation allocates a root bitmap, makes both children at each split, keeps the alternatives alive, releases each parent, enumerates the final leaf and releases the remaining masks. The surviving path and alternatives are checked to form a complete, non-overlapping partition.

Widths range from 512 bytes to 125 MB. The largest width is one bit over a billion document positions. Thirteen stored planes and the live masks reached about 3.50 GB of process memory. That is a measured buffer set, not the full 256-plane index. The whole-index calculation still has to include every stored plane, packed document and ID.

The larger buffers showed median split costs of roughly 0.34–0.37 ns per physical word in this standalone loop. At 256 dimensions, contiguous scoring took about 27 ns per document at one and four million rows. Small postings cost more per row because preparation and selection are less amortized. Compiler context and the surrounding query still matter; these observations are not a universal instruction-price table.

### 10.2 A scored row does not always have the same price

A second operation reproduces the leaf-scoring loop: read a bitmap word, find its set bits, fetch those document codes and IDs, score them, and maintain top  $K$ . It uses the same scorer as the sequential loop. One run reuses a candidate bitmap after warmup. Another prepares a different bitmap before each measured trial.

| 16M-document buffer      | Reused candidates | Changing candidates |
|--------------------------|-------------------|---------------------|
| About 1,958 scored rows  | 0.159 ms          | 0.455 ms            |
| About 62,500 scored rows | 4.400 ms          | 8.271 ms            |

The selection densities are the same; the selected IDs are not. Candidate generation is outside the timer, but its effect on cache history remains. This shows why the scan’s cost per row should not simply be copied into B’s scattered leaf scores. It does not directly measure cache-miss counters or establish fixed CPU frequency.

A three-term fit used a fixed cost, physical bitmap words and scored rows. It was fitted on 1M and 4M pools and checked on a 16M pool. The reused-candidate model underestimated one check by 45.2%. With changing candidates, the worst error was still an underestimate of 27.6%. Both fits remain saved. The largest row pool was about 640 MB; it was not a 40 GB row-store test.

Do not add the enumeration-only leaf timer to the gathered-leaf timer: the latter already includes enumeration. This would double-count one stage. Similarly, a fitted zero intercept is not evidence that score-table preparation is free; other fitted terms can absorb that cost.

### 10.3 Use a compact independent reference for larger native runs

The native check uses 1M, 2M and 4M binary documents, then a separate 8M run. It keeps the constructed strong/weak query distribution. Packed codes are generated in small chunks and the first million rows are checked against the earlier document pool.

An independent reference avoids expanding those rows into a huge float matrix. For each query, count the strong and weak mismatches directly from the packed bits. With magnitudes  $a$  and  $\epsilon$ , the score is

$$S = a(m - 2e_{\text{strong}}) + \epsilon(d - m - 2e_{\text{weak}}).$$

Sort by this score and then ID. Small tests compare that calculation with the earlier direct float sum, including a dimension count that does not fill the final packed word. All complete native calls in the scaling check matched the independent reference.

For B, the leaf threshold grows with the corpus:  $L = 160N/10^6$ . This preserves the intended 13 strong-coordinate cuts. With fixed depth and density, split words, leaf words and scored rows all grow approximately in proportion to  $N$ . A simple complete-call model is then

$$\hat{T}(N) = a_0 + b_0N.$$

The fixed term covers work that does not grow with the corpus; the slope combines the work that does. This is a conditional model of the specified path, not a reason to keep  $L = 128$  while increasing  $N$  indefinitely.

### 10.4 Two model mistakes, and how the checks exposed them

The first fit minimized percentage error over individual queries but was used to predict a median. Fast queries received more weight, which was the wrong target. For example, workloads with times  $[1, 10, 10]$  and  $[2, 20, 20]$  have medians 10 and 20. Weighting each query by inverse time favors the fast minority and predicts the wrong median. The repaired fit uses one median per

corpus size. A test fails for the mixed workload under the old calculation and stays correct for a uniform workload. The original fit is preserved.

A linear row term was also insufficient for C’s small postings. The shared result heap has visible selection cost when only a few hundred documents are scored. Under independently ordered continuous scores, document  $i$  enters the current top  $K$  with probability  $K/i$ . The expected number of replacements after the first  $K$  rows is

$$\sum_{i=K+1}^F \frac{K}{i} \approx K \ln(F/K).$$

For fixed  $K$ , a useful approximation is

$$\hat{T}_C(F) = a_C + b_C F + c_C \ln(\max(1, F/K)). \quad (75)$$

Binary ties and key ordering change the exact replacement count, so the logarithm is a model term, not an exact counter. A large contiguous run supplies the row price; the native 1M/2M measurements fit the remaining terms. This corrected the small-posting prediction without changing the reference ranking or search implementation.

The revised 4M check is diagnostic because its results had already been inspected. The coefficients were frozen before the 8M run. That run checks transfer to a larger corpus using the same prescribed query workload; it is not a new-query generalization test.

| Query support | Method | Measured ms | Predicted ms | Error |
|---------------|--------|-------------|--------------|-------|
| Fixed         | A      | 197.029     | 203.016      | +3.0% |
| Fixed         | B      | 0.854       | 0.884        | +3.6% |
| Fixed         | C      | 0.0481      | 0.0518       | +7.7% |
| Changing      | A      | 196.664     | 197.816      | +0.6% |
| Changing      | B      | 0.900       | 0.859        | −4.5% |
| Changing      | C      | 202.767     | 207.086      | +2.1% |

All six checks met the earlier 20% criterion. This supports the stated model over the checked sizes. It does not turn the largest observed error into a guaranteed error bound at a billion rows. The earlier failed fits remain part of the evidence.

## 10.5 Check a parameter suggested by the distribution

At one million rows, the ideal 13-bit group has mean  $\lambda \approx 122$  and standard deviation about 11. A leaf size of 128 can force another cut when the strong-matching group is slightly larger than average. A rough upper margin  $\lambda + 3\sqrt{\lambda}$  is about 155; choosing 160 avoids many unnecessary weak-coordinate cuts. The preceding 12-bit group has mean about 244, so it will usually still be split. These are distribution-based settings to check, not universal thresholds or a guarantee from a normal approximation.

The bounded follow-up adds 16,384 clusters, neighboring key widths and leaf sizes, including 160 for the changing-coordinate case. It remeasures prior choices rather than comparing fresh timings against old ones. The first 40 generated queries become the tuning set; query IDs 40–59 are used only after the new choices are frozen. Document and reference prefixes are checked before reusing any router.

At 1M rows and the 99% target, the changing-coordinate follow-up selected B with one cluster and  $L = 160$ . It measured 0.1446 ms at 100% binary recall. A measured 24.9235 ms at 99.95%: a paired speedup estimate of 172.3, with 95% interval 169.1–177.2. C chose a 14-bit key and measured 14.6301 ms at 100%, but its paired speedup interval against A included one. That point estimate alone does not establish a C advantage.

The fixed-coordinate follow-up selected a 16,384-cluster, two-probe scan and a global 13-bit key index. Both reached 100% recall: A measured 0.0140 ms and C 0.0128 ms. Their small gap again concerns closely related candidate selection, not a general dominance theorem.

## 10.6 Why the key method’s median can change abruptly

In the changing-support model, a fixed  $h$ -coordinate key captures a random number  $S_h$  of the  $m$  strong coordinates. Its distribution is hypergeometric:

$$\Pr(S_h = s) = \frac{\binom{m}{s} \binom{d-m}{h-s}}{\binom{d}{h}}. \quad (76)$$

For  $d = 256$ ,  $m = 13$ , the probability of capturing none is 52.75% at  $h = 12$ , 49.94% at  $h = 13$ , and 47.27% at  $h = 14$ . A small width change can move a sample across the halfway point between queries that score nearly all rows and queries that can discard about half.

Under the strong-score gap and the additional condition that weak-key bounds cannot prune individual weak patterns, the work for a query capturing  $s$  strong coordinates is approximately

$$H = 2^{h-s}, \quad F = N2^{-s}.$$

The expected scored fraction is  $\sum_s \Pr(S_h = s)2^{-s}$ : about 71.30% at  $h = 13$  and 69.44% at  $h = 14$ . The mean work changes smoothly even when a small sample’s median changes sharply. This is another reason to keep query-level uncertainty and the full latency distribution. The condition is about the stated constructed distribution, not arbitrary embeddings or IVF.

## 10.7 A shortest-key control on new real queries

The original real grid started with four-bit keys. A final control gives C a one-bit key and A’s routing layout, with no candidate or lookup limit. It keeps A and B’s frozen settings and uses 100 further query IDs, excluding all 500 previously used IDs. No parameter is changed based on this control’s outcomes.

A measured 9.541 ms, B 9.855 ms and C 10.545 ms. All three reached 98.95% binary recall, narrowly below the requested 99%. The miss stays in the report; no tolerance is added to turn it into a pass. There is no supported alternative speedup in this check. The original 200-query evaluation remains a separate result, where its conservative choices met their sample targets. A finite-sample selection rule is not a promise that every later query sample will reach the same mean.

## 10.8 Measure the text query as one request

The final pipeline check runs actual text through the cached local Nomic model on MPS, applies the same normalization, routes the resulting vector and calls the native search. Batch size is one.

Each sequence shape is warmed first; model loading, index building and reference creation are outside the request timer. References are computed afterward from the actual encoded vectors.

| Method | Encode median | Retrieve median | Total median | Total p95 |
|--------|---------------|-----------------|--------------|-----------|
| A      | 6.488 ms      | 9.406 ms        | 16.313 ms    | 19.201 ms |
| B      | 6.500 ms      | 9.651 ms        | 16.303 ms    | 19.217 ms |
| C      | 7.543 ms      | 11.201 ms       | 18.912 ms    | 21.898 ms |

These are 20 existing independent-query IDs, three blocks and two repeats, with the earlier frozen settings. All reached 99.45% binary recall on this sample. The tiny A/B difference does not establish a speedup. Each total is measured per request: its median is not the sum of the two stage medians. The process lifetime peak was about 3.67 GB including reference construction; MPS driver allocation was about 1.08 GB and may overlap RSS, so those numbers are not simply added.

This text check applies to the real Nomic queries. The constructed strong-coordinate vectors do not come from this encoder, so its time must not be added to those examples and called a measured end-to-end sparse-embedding pipeline.

## 11 Bring the formulas together

The three methods share query embedding, preparation, routing, filtering, and the optional final reranker. Their difference is the work used to obtain candidates from the selected pool. Keeping those shared costs visible lets us compare both the index work and the complete query.

### 11.1 Latency, one method at a time

Using the earlier definitions,

$$T_A = T_{\text{common}} + P c_{\text{range}} + N_{\text{elig}} G c_{\text{lut}} + H_s(N_{\text{elig}}, R) c_{\text{cheap}} + T_{\text{rank}}(R) + T_{\text{finish}}(R'), \quad (77)$$

$$T_B = T_{\text{common}} + T_{\text{order}} + T_{\text{roots}} + \sum_j s_j W_j c_{\text{split}} + \sum_j \ell_j W_j c_{\text{leafword}} \\ + V c_{\text{node}} + F G c_{\text{lut,leaf}} + H_s(F, R) c_{\text{cheap}} + T_{\text{rank}}(R) + T_{\text{finish}}(R'), \quad (78)$$

$$T_C = T_{\text{common}} + T_{\text{keyprep}} + E c_{\text{enum}} + C^{\text{copy}} c_{\text{copy}} + \sum_j (H_j c_{\text{lookup}} + F_j c_{\text{post}}) \\ + F G c_{\text{lut,posting}} + H_s(F, R) c_{\text{cheap}} + T_{\text{rank}}(R) + T_{\text{finish}}(R'). \quad (79)$$

This C expression follows the implemented shared stream:  $E$  counts its enumeration work once, and  $C^{\text{copy}}$  counts its mask-copy work once. Cluster sums cover actual directory lookups and postings.  $F = \sum_j F_j$ , and the current scorer evaluates all  $G$  groups per candidate. A tail-only or full-key score-reuse implementation would substitute its actual group count; those savings are not credited to the measurements here.

## 11.2 An index speedup is not the same as an end-to-end speedup

If two configurations have identical embedding and final-stage costs, write their shared time as  $T_0$ . Replacing index time  $t_A$  by  $t_B$  gives speedup

$$\frac{T_0 + t_A}{T_0 + t_B},$$

not  $t_A/t_B$ . Even removing index time entirely cannot exceed  $1 + t_A/T_0$  when  $T_0 > 0$ . If candidate counts or final-vector fetches differ, those costs are no longer shared and must be evaluated separately. This is why cached-query index timings and complete text-query timings should both be reported rather than substituted for one another.

## 11.3 Persistent storage

The earlier reference layouts explain logical payload. For the implemented native cores, write the allocated terms explicitly:

$$M_{A,\text{core}} = M_{\text{codes},\text{cap}} + M_{\text{IDs},\text{cap}} + M_{\text{other},A}, \quad (80)$$

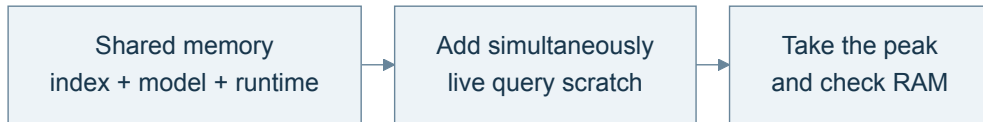
$$M_{B,\text{core}} = M_{\text{codes},\text{cap}} + M_{\text{IDs},\text{cap}} + M_{\text{planes},\text{cap}} + M_{\text{other},B}, \quad (81)$$

$$M_{C,\text{core}} = M_{\text{codes},\text{cap}} + M_{\text{IDs},\text{cap}} + \sum_j (U_j b_{\text{node}} + L_j b_{\text{ptr}}) + M_{\text{other},C}. \quad (82)$$

“cap” means allocated array capacity. The remaining terms cover bucket objects, the outer label map and allocator bookkeeping not already counted. C’s IDs are its postings; there is no second  $N$ -row posting permutation. Its hash-node and bucket-pointer terms use the separate chaining layout in Equation (67).

The plane payload is  $db_w \sum_j \lceil n_j/w \rceil$ ; allocated capacity is at least that payload. Storage sums run over all clusters, not just the probed ones. Add any retained routing index, filters and fine-ranking vectors to the native core to obtain the resident index. Input arrays kept only during construction belong to the build peak, not the serving-state sum.

## 11.4 Peak query RAM and the serving budget



**Figure 10:** Persistent index data, runtime reserve and simultaneous scratch are separate terms. Sequential stage buffers need a lifetime analysis.

The search-stage scratch terms are

$$Q_{A,\text{search}} = Q_{\text{base}} + Q_{\text{scan workspace}}, \quad (83)$$

$$Q_{B,\text{search}} = Q_{\text{base}} + Q_{\text{order}} + \max_t [\gamma_B Q_{B,\text{frontier}}(t) + Q_{\text{current}}(t) + Q_{\text{children}}(t)], \quad (84)$$

$$Q_{C,\text{search}} = Q_{\text{base}} + Q_{\text{keyorder}} + Q_{\text{selected}} + \max_t [\gamma_C Q_{\text{enum}}(t) b_{\text{state}}]. \quad (85)$$

C has one queue for the common key coordinates, not one queue per probed cluster.  $Q_{\text{keyorder}}$  stores the sorted coordinate indices;  $Q_{\text{selected}}$  stores the selected bucket pointers.  $\gamma_C Q_{\text{enum}}$  accounts for queue capacity rather than only live entries. The candidate heap is already in  $Q_{\text{base}}$ , so it is not added again. The current C implementation reads posting ranges directly and allocates no separate gathered-document buffer.

At the Python boundary, native result vectors and output arrays may coexist. Include that interface workspace when evaluating the full request peak. Whole-process observations include it even when individual interface allocations are not separately instrumented.

Small iterator state is included in workspace terms. Embedding and final reranking can have larger scratch than search, so use Equation (12) to obtain the full query peak. With resident index size  $M_{m,\text{resident}}$  for method  $m$ , the RAM constraint is

$$M_{m,\text{resident}} + M_{\text{model}} + M_{\text{runtime}} + \max_t \sum_{u \in \mathcal{A}(t)} Q_{m,u}(t) \leq M_{\text{RAM}}. \quad (86)$$

$M_{\text{model}}$  is the shared resident embedding-model memory if encoding runs locally; it is zero in this memory domain when an external service performs encoding. It is not multiplied by the number of queries.

For a conservative bound, replace the sum by  $JQ_{m,\text{peak},\text{max}}$ . A loose upper bound that exceeds RAM does not prove that a tighter implementation cannot fit. An observed peak below the limit does not prove that every possible query fits. These are different claims.

The native measurements use the allocation choices in Equation (67): C reorders codes and IDs together, so its extra posting permutation is absent. Its directory uses allocated nodes and bucket pointers. Use that layout when connecting the general formulas to the measured results.

## 11.5 A compact comparison table

|                         | A: full scan                              | B: bitplanes                                       | C: key enumeration                                           |
|-------------------------|-------------------------------------------|----------------------------------------------------|--------------------------------------------------------------|
| Additional index        | none beyond common layout                 | $db_w \sum_j W_j$                                  | key directory; IDs already counted                           |
| Principal work          | every eligible code                       | split masks, leaf masks, full scores, frontier     | empty and occupied keys, enumeration, postings, full scores  |
| Variable counts         | $N_{\text{elig}}$                         | $s_j, \ell_j, f_j, V$                              | $H_j, F_j, E, Q_{\text{enum}}^{\text{max}}$                  |
| Largest scratch concern | selection or batch score buffers          | many full-width pending masks                      | pending flip sets and interface outputs                      |
| Local exactness         | scan all eligible rows                    | valid bounds, sufficient traversal, correct ties   | full-key weighted order or a valid full-score bound          |
| Approximation control   | routing probes; candidate/rerank boundary | node budget, visit policy, earlier routing         | prefix width, candidate or key budget, earlier routing       |
| What must be measured   | kernel cost, routing quality              | skipped scores versus bitmap/queue cost and recall | empty-key volume versus useful candidates, timing and recall |

## 11.6 The variables most likely to move the tradeoff

**Documents per physical cluster,  $n_j$ .** A grows roughly linearly with eligible row count. In B, a larger cluster also widens every active bitmap. In C, more documents per key can reduce the number of empty probes needed, while increasing posting and scoring work.

**Binary dimension,  $d$ .** Longer codes increase score terms and stored bits. B may gain more from avoiding full scores, but also stores more planes and can traverse more coordinates. For full-key C, the number of possible keys grows exponentially. For short-key C,  $h$  and  $d$  are separate parameters.

**Probes,  $P$ .** More clusters can recover missed neighbors. They also increase eligible rows, root masks, active key streams and candidate competition. Cluster sizes and the query’s routing behavior matter, not just the number  $P$ .

**Leaf size and node budget.** A small leaf threshold can trade scoring for more mask work. A finite node budget caps visits but does not specify how many useful leaves are reached. The best choice depends on the query and data, not a general monotonic recall formula.

**Lookup width and candidate target.** A short key has denser postings and a smaller key space, but leaves more score information out. A longer key can improve candidate discrimination and simultaneously make empty-key enumeration costly. A strong prefix can improve recall; that does not remove the enumeration cost.

**Kernel and memory system.** SIMD, cache residency, batch size, alignment, layout and thread count change the effective coefficients. Keep them fixed for an algorithm comparison, or report their changes as a separate experiment.

## 11.7 Choosing configurations at fixed N and RAM

Let  $\theta_m$  contain the chosen method’s tunable parameters and let  $r_0$  be a recall target. Choose the latency objective explicitly, for example median or p95 across queries. A valid optimization statement is

$$\theta_m^* \in \arg \min_{\theta_m} \text{Quantile}_p[T_m(q; \theta_m)] \quad (87)$$

subject to the RAM condition in Equation (86) and

$$\mathbb{E}_q[\text{Recall@ } K(q; \theta_m)] \geq r_0. \quad (88)$$

The expectation specifies an average-recall target. A requirement that almost every query reach  $r_0$  is stronger and needs a different probability constraint.

**Listing 14:** A defensible finite-grid experiment.

```

ChooseConfiguration(corpus, queries, fixed_reference, RAM_limit, targets):
    build or cache every configuration's required index
    compute exact reference results once for each query
    on tuning queries:
        for every configuration in the declared grid:
            measure latency, quality, work counts and peak RAM
            keep failures and empty returns in the record
    choose qualifying configurations for each target
    on separate test queries:
        evaluate those choices without retuning
    report lowest tested latency only where the target and RAM checks pass
    distinguish measured results from untested model predictions

```

The parameters are often integers: cluster count, probes, lookup bits, and visited nodes. A small declared grid handles these choices directly. Differentiation can simplify a smooth part of the cost model, but it does not determine whether the recall constraint is satisfied. For example, a model  $T(C) = aC + bNP/C$  has a stationary point at  $C = \sqrt{bNP/a}$  when  $a, b > 0$  and  $P$  is held fixed. Check neighboring feasible integers and memory use. If the probes required for the recall target change with  $C$ , that derivative no longer solves the complete problem.

Choose a setting using calibration and tuning queries, then check the frozen choice on separate queries. Optimizing an inaccurate recall model can select the configuration where that model makes its most optimistic error. At 99% recall, a predicted 99% is insufficient; report actual query-level quality and its uncertainty. The existing normal prediction's real-data errors are too large to establish such a winner.

If  $d$  is also varied, keep the quality reference fixed across choices, for example the same long-vector top  $K$  or the same relevance labels. Recomputing a different binary ground truth for every  $d$  makes a tiny representation look perfectly accurate against its own changed task.

## 11.8 What can be calculated at a billion documents?

The native cost models were checked through 8M rows. The bitmap profile reaches the physical width of a 1B-row mask, but no complete 1B index was built or queried. The following numbers are conditional forecasts using the same 256-bit score, top 100, one CPU thread and the stated strong-coordinate distribution. A different machine or access pattern can change the prices.

Start with memory, because it can reject a configuration without estimating its latency. For the current layout at  $N = 10^9$  and  $d = 256$ ,

$$M_{\text{codes}} = 32 \text{ GB}, \quad M_{\text{IDs}} = 8 \text{ GB}.$$

A and C need at least 40 GB before directories, containers and query buffers. B retains a second transposed copy, adding at least 32 GB. Therefore all three fail a 32 GB limit, and B also fails a 64 GB limit. Changing probes does not remove stored documents. These rejections apply to the current in-memory layouts; they are not claims about every compressed or disk-based design.

The planning figures below allow up to twice the logical row-ID capacity, a small directory and container allowance, and a configurable 1 GB runtime reserve. The reserve is an allowance, not measured runtime memory. Estimated fits are different from the payload lower bounds that

prove rejection. Building an index from full input copies can require more RAM than serving it.

| 1B-row path                          | Payload lower bound | Planned query RAM | 64 GB budget |
|--------------------------------------|---------------------|-------------------|--------------|
| A, known strong-prefix routing       | 40 GB               | about 49 GB       | Modeled fit  |
| C, global 13-bit key                 | 40 GB               | about 49 GB       | Modeled fit  |
| B, one global cluster                | 72 GB               | about 83 GB       | Rejected     |
| B, same prefix route, scan-like leaf | 72 GB               | about 81 GB       | Rejected     |

All these planning figures fit a 1 TB budget. The model does not claim that a 1 TB machine has the M5 Max’s exact memory behavior. The measured row buffers were much smaller than a 40 GB store.

## 11.9 When the important coordinates stay in one prefix

With thirteen fixed strong coordinates, the ideal group contains an expected  $10^9/2^{13} \approx 122,070$  rows. Under the independent-bit assumption, a binomial calculation controls the chance of having fewer than 100. A Chernoff bound gives a probability below  $10^{-26464}$  in this example. This extremely small bound belongs to the stated distribution; it is not an embedding-accuracy guarantee.

A can route directly to that group and scan it. C can look up the same key and score its posting range. Their modeled costs are both roughly 3 ms. The data being read after routing is only a small part of the corpus, although all codes and IDs are still stored.

B must get the same opportunity: it can use that prefix routing and choose a leaf large enough to scan the selected group. An all-ones leaf-mask profile shows a similar scoring cost. Thus B’s fallback also models at roughly 3 ms, with substantially more index storage. Forcing B to traverse the entire billion-row bitmap would be an unfair parameter choice in this case. The model cannot distinguish a few-percent timing difference here. A or C is the simpler memory choice; this is not a proof that one API name is always faster.

### 11.10 When the important coordinates change with the query

A fixed prefix no longer supplies the same filter. The measured B path can select its columns from the actual query. Keeping thirteen cuts and scaling the leaf threshold to 160,000 gives an expected final set of about 122,070 rows. Its mask payload at the last split is

$$(13 + 2) \left\lceil 10^9/64 \right\rceil \times 8 = 1.875 \text{ GB}.$$

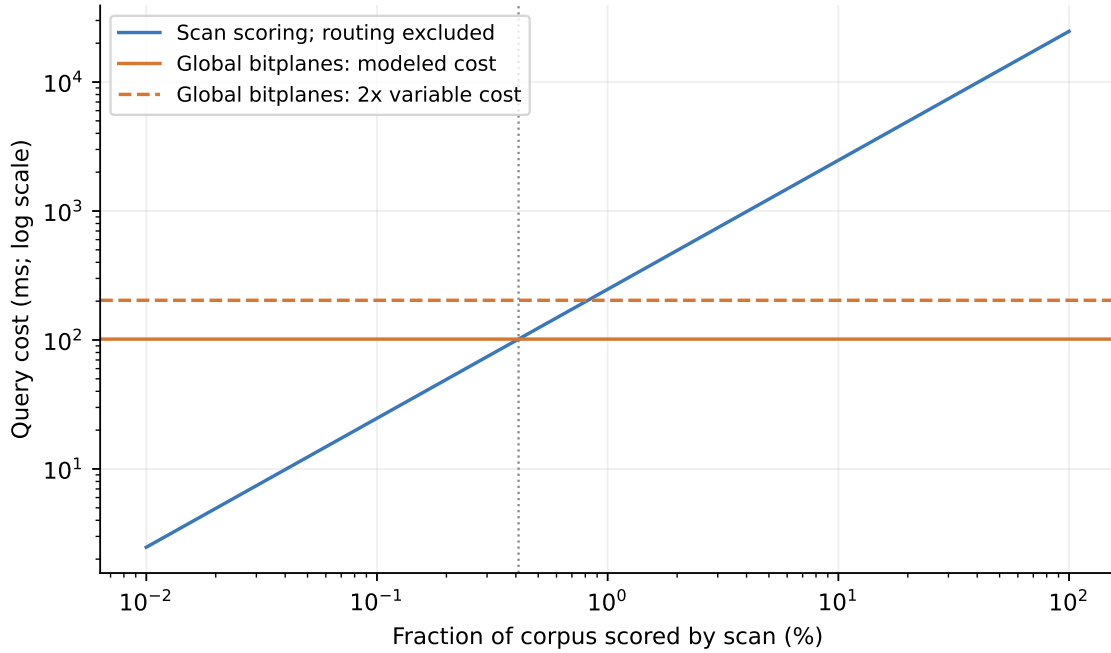
The native size model estimates about 0.10 s for this global B path. Doubling its variable cost gives about 0.20 s. This doubling is a sensitivity scenario, not a confidence interval or a promised bound. The global scan model is about 25 s.

That comparison does not establish that B beats every possible IVF configuration. Let  $f$  be the fraction of the corpus A actually scores, and let  $T_{\text{route}}$  be the time needed to obtain those candidates. A’s approximate cost is

$$T_A \approx T_{\text{route}} + fNc_{\text{scan}}.$$

Ignoring routing time gives A the most favorable comparison. At the current prices, its scanned fraction must be below about 0.41% to undercut the 0.10 s B point, or below about 0.82% if

B's variable cost doubles. It must still meet the recall target at that fraction. If routing already produces a sufficiently small, accurate pool cheaply, scanning can win. If it cannot, a global or less heavily clustered B path is a candidate worth testing.



**Figure 11:** A conditional break-even calculation at 1B rows with sufficient RAM. The scan line excludes routing time, favoring A. B is one fixed global operating point, not a retuned curve. Routing recall and the concentrated-query assumption must be checked separately.

B can also use a router. In that case, substitute the physical cluster widths and visited work into the earlier sum over clusters. The graph does not solve that larger optimization problem or assume a relevant document is always in the probed clusters.

For C, the number of strong coordinates actually captured by its key matters. Under Equation (76) and the stated bound conditions, a query capturing none can still require scoring nearly all rows; capturing one leaves roughly half. A fixed short key can therefore remain expensive even when it has many occupied entries and few empty lookups. It is not enough to argue that finding a nonempty key is cheap.

### 11.11 The usable conclusion

The completed tests establish three different outcomes within their declared spaces:

- On the real Nomic sample, no alternative speedup was established. Even a one-bit key control did not improve latency. Recall targets must retain their sampling uncertainty.
- With a fixed strong prefix, direct routing and integrated key lookup are closely related and both avoid most scoring. C had a small measured implementation advantage in the controlled runs.
- With changing strong coordinates, B's query-dependent column order produced a large, independently checked advantage in the constructed data. Increasing the leaf size with corpus size preserved a useful path; keeping a tiny fixed leaf would not.

At a billion rows, memory lower bounds remain exact for the layout, while latency and the best routing parameters remain conditional forecasts. The formulas give operating points and break-even requirements to test. They do not turn a finite grid, or the name of an embedding model, into a universal optimum.

The current calculation is `experiments/project_costs.py`; its inputs and all scenarios are saved in `results/projected-cases-2026-09-11/`. The earlier `reports/search-math/numerical-cases/calculate.py` is preserved as a historical assumed-cost model. Its extra-posting and hash-slot layout must not be substituted for the measured native C layout.

#### What the comparison tells us

A pays to score the selected pool. B pays to discover which scores can be avoided, and dense bitmaps can make that discovery expensive. C pays to discover which keys contain useful candidates, including empty keys and queue work. Each can be exact under a suitable stopping rule. Which is fastest at a recall target is determined by the data, the representation, the visited work, the memory layout and the hardware. The exact small examples explain how recall can be calculated. The measured prediction errors show where the assumptions need more evidence before they can guide a real configuration choice.

## A Computing the expected top-K count at each distance

The count  $G_K(p)$  in Equation (46) can be evaluated directly as

$$G_K(p) = \sum_{x=0}^N \min(K, x) \binom{N}{x} p^x (1-p)^{N-x}.$$

For large  $N$ , computing the binomial coefficients and powers separately can overflow or lose small probabilities. A shorter expression follows from a counting identity. For an integer  $x \geq 0$ ,  $\min(K, x)$  counts how many thresholds  $1, 2, \dots, K$  it reaches:

$$\min(K, x) = \sum_{j=1}^K \mathbf{1}[x \geq j].$$

Taking expectations on both sides gives

$$G_K(p) = \sum_{j=1}^K \Pr(X \geq j), \quad X \sim \text{Binomial}(N, p). \quad (89)$$

Use a numerically stable binomial survival function for these probabilities. The ordinary small-example checker uses the direct sum; the simulation checker uses this alternative. Their agreement supplies a check across two different calculations.

For the distance model,  $\sum_m w_m = G_K(1) - G_K(0) = K$ . Thus the weights in Equation (47), divided by  $K$ , form a probability distribution over the distance of a uniformly chosen true top- $K$  row. Given that row's distance, symmetry makes its error positions uniform. The ID rule chooses among tied rows without favoring one set of error positions. This justifies multiplying  $w_m$  by the hypergeometric survival probability.

The argument would change if ties deliberately favored matching prefixes. It would also change if we stopped candidate collection after a corpus-dependent quota. These are changes to the selection policy, not numerical errors in the fixed-radius formula.

**Listing 15:** Expected top-K recall for a fixed prefix radius.

```
ExpectedTopKRecall(N, d, h, radius, K):
    previous_slots = 0
    expected_found = 0
    for distance in 0 .. d:
        p = F_d(distance)
        slots = sum(Pr(Binomial(N, p) >= j) for j in 1 .. K)
        rows_at_distance = slots - previous_slots
        expected_found += rows_at_distance * g_h_radius(distance)
        previous_slots = slots
    return expected_found / K
```

## B A finite-sample view of prefix recall

The population-tail approximation is not the only mathematical definition available. This appendix shows what an exact finite- $N_*$  calculation would require, under independent, identically distributed document score pairs and no score ties. It is useful for understanding why knowing only a correlation is insufficient.

For a particular document with full score  $s$  and prefix score  $t$ , classify another document into four events:

$$\begin{aligned} p_{11}(s, t) &= \Pr(S > s, T > t), & p_{10}(s, t) &= \Pr(S > s, T \leq t), \\ p_{01}(s, t) &= \Pr(S \leq s, T > t), & p_{00}(s, t) &= \Pr(S \leq s, T \leq t). \end{aligned}$$

Among the other  $N_* - 1$  documents, let  $(a, b, c, e)$  count these categories. A multinomial distribution counts how many independent documents land in each of the four categories, using those probabilities. The chosen document belongs to the full top  $K$  when  $a + b \leq K - 1$ , and belongs to the prefix top  $R$  when  $a + c \leq R - 1$ . Therefore, with  $e = N_* - 1 - a - b - c$ ,

$$P_{K,R}(s, t) = \sum_{\substack{a, b, c \geq 0; \\ a+b \leq K-1; \\ a+c \leq R-1 \\ a+b+c \leq N_*-1}} \frac{(N_* - 1)!}{a!b!c!e!} p_{11}^a p_{10}^b p_{01}^c p_{00}^e. \quad (90)$$

If  $F_{S,T}$  is the joint distribution of one document's scores, every document has the same event probability under the identical-sampling assumption. Summing over all  $N_*$  documents and dividing by  $K$  gives

$$\mathbb{E}[\text{Recall@ } K] = \frac{N_*}{K} \int P_{K,R}(s, t) dF_{S,T}(s, t). \quad (91)$$

This assumes the candidate stage returns exactly the prefix top  $R$  and the final stage uses the same full score. Retrieving whole tied prefix keys changes that candidate rule; ties can be handled by augmenting the ordering with document IDs or by explicitly integrating their selection probabilities.

Equation (91) is much harder to evaluate than the conditional-normal approximation. More importantly, it still needs  $F_{S,T}$ . Matryoshka training, a code length, a cluster count and a RAM budget do not uniquely specify that distribution. Exact mathematics can organize unknown inputs; it cannot infer them from the algorithm's name.

## C Checklist for an honest numerical implementation

1. Fix the score and tie rule. Keep original-float, truncated-float, binary-score and human relevance references distinct.
2. Check the score identity on small vectors and enumerate the actual top  $K$ .
3. Check that the weighted-subset stream emits every subset once, in nondecreasing penalty, and processes the final boundary tie before claiming exactness.
4. Count split words and leaf words separately. Use each cluster's physical width, including padding; do not substitute the number of active documents for the bitmap length.
5. Record the ownership of codes, external IDs, posting references and key storage. Any shared or reordered array must be stated before removing its bytes from a formula.
6. Count temporary state at its lifetime peak. A queue-capacity bound and an observed allocation peak are different quantities.
7. Keep key attempts separate from occupied keys and returned postings. Expected first hit is not the same threshold as finding  $R$  candidates.

8. Price centroid float arithmetic separately from binary score lookups. Charge selection, allocation, copying and gathers where the implementation performs them.
9. Use either a calibrated combined cost or a disaggregated operation model for each block. Do not count the same memory and selection work in both.
10. Test statistical predictions on queries not used to estimate their parameters. Evaluate recall after the actual candidate and tie rules, not merely a probability of finding some key.

## D Floating-point arithmetic and exactness

The score and bound proofs use real arithmetic. A numerical implementation must ensure its computed upper bounds remain conservative under rounding. The local C++ reference includes a rounding allowance for bound pruning and a deterministic row-ID tie order. A report should not claim exactness merely because an unguarded floating-point comparison resembles the algebra.

This does not require inventing a collection of hypothetical production patches. It requires checking the numerical path used by the implementation and testing it against a full reference scan on the same inputs. The small examples and archived candidate checks provide such evidence for their stated scopes; they do not prove every future implementation correct.

## E Optional expansion of the embedding cost

The main formulas keep  $T_{\text{emb}}$  and  $Q_{\text{emb}}$  as measured model-specific functions. If a more detailed encoder estimate is needed, its architecture must be another set of inputs, not an unexplained constant.

For an illustrative standard dense Transformer encoder, let  $n_t$  be query tokens,  $L_e$  the number of layers,  $m$  hidden width,  $f$  feed-forward width, and  $H_a$  attention heads. For one layer with ordinary two-matrix feed-forward computation, multiply-add contributions are

$$U_{\text{layer}} = 4n_t m^2 + 2n_t^2 m + 2n_t m f. \quad (92)$$

The first term comes from the query, key, value and attention-output projections. The second comes from attention-score and attention-value matrix products. The third comes from the feed-forward expansion and contraction. Activation, normalization and softmax operations add separate elementwise terms. A different block, gated feed-forward network or sparse attention requires different counts.

A corresponding parameterized time model is

$$T_{\text{emb}} = T_{\text{tokenize}} + L_e (U_{\text{layer}} c_{\text{enc,mac}} + U_{\text{elementwise}} c_{\text{enc,elt}}) + T_{\text{pool/project}}.$$

These effective coefficients depend on batching, device kernels, precision and memory behavior. Alternatively use the resource bound in Equation (19), without adding it on top of coefficients that already include the same work.

A non-fused attention implementation may materialize  $H_a n_t^2$  attention values. With bytes per activation  $b_a$ , an explicit scratch model can be written

$$Q_{\text{emb}} = b_a (\eta_m n_t m + \eta_f n_t f + \eta_a H_a n_t^2) + Q_{\text{kernel}},$$

where the  $\eta$  coefficients count simultaneously live buffers, not time. Fused attention can avoid materializing the square attention matrix, so its chosen buffer counts differ. Layer workspaces can be reused during inference; multiplying a per-layer peak by  $L_e$  is not a general inference-RAM formula. Resident model weights are shared and counted separately.

A Matryoshka prefix normally reuses an already computed embedding. Halving the search prefix  $d$  does not automatically halve the encoder’s layer count or hidden width. It reduces the subsequent representation and search work unless the actual encoder implementation also changes.

## F Reproduction and parameter ledger

The editable source is `report.tex`, with chapter files in `sections/`. Mermaid sources and rendered diagrams are in `figures/`. The build instructions are in the adjacent `README.md`. The arithmetic checks are implemented in `verify_math.py`. They validate the worked examples, exact subset ordering, binomial conditional-recall example, word widths, selected storage identities, and quoted measured rows. The statistical evidence is preserved with its source hash.

The following ledger identifies the inputs a future calculator must obtain rather than infer from one document count.

| Input family         | Parameters or counts                                  | How to obtain them                                               |
|----------------------|-------------------------------------------------------|------------------------------------------------------------------|
| Representation       | $D, d, d_r, d_f$ , preparation and score rule         | fixed experimental definition; embedding/model metadata          |
| Retrieval target     | $K, R$ , filter, reference score, tie rule            | fixed task definition                                            |
| Cluster layout       | $C, n_j, \mathcal{P}(q), e_j$                         | built index and per-query routing/filter traces                  |
| Storage widths       | $w, b_w, b_{id}, b_o, b_c, b_f, b_s$                  | actual types, packing and alignment                              |
| Baseline scoring     | $g, G, H_s, c_{lut}$                                  | source code, work counters and kernel measurements               |
| Bitplane search      | $L, B_{node}, s_j, \ell_j, f_j, V$                    | configuration plus query traces; not budget alone                |
| Bitplane RAM         | frontier masks, current/child buffers, capacities     | allocation/capacity measurements or an explicit bound            |
| Key index            | $I_h, h_j, U_j, \alpha, b_k, b_{ref}$                 | chosen lookup coordinates, index build and slot layout           |
| Key search           | $H_j, F_j, E_j, Q_j^{\max}$ , stopping rule           | actual key enumeration, posting and frontier traces              |
| Prefix-trie variant  | $Z_j, b_{trie}, u_j$ , posting ranges                 | actual stored nodes, path and range visits                       |
| Hardware             | compute rates, bandwidth, cache regime, SIMD, threads | calibrated for the actual workload and machine                   |
| Optional final stage | fetch policy, $R', u_f, d_f$ , rerank score           | execution pipeline and storage-residency decision                |
| Statistical model    | query/data distribution, covariance, score tails      | declared synthetic assumptions or measured and checked estimates |
| Serving RAM          | $M_{resident}, M_{model}, M_{runtime}, J, Q_{peak}$   | ownership/lifetime analysis and process measurements             |

A numerical example can assign values to these parameters. The equations themselves remain

parameterized, so changing the workload changes the inputs instead of requiring a different explanation of the algorithm.

## References

- [1] Exa Team. *How we built a web-scale vector database*. 17 December 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022, revised 2024. <https://arxiv.org/abs/2205.13147>.
- [3] M. Norouzi, A. Punjani, D. J. Fleet. *Fast Exact Search in Hamming Space with Multi-Index Hashing*. 2013, revised 2014. <https://arxiv.org/abs/1307.2982>. This concerns Hamming search; it is not a weighted-search guarantee for the methods here.
- [4] Local project evidence. `cpp/index.cpp`, `cpp/index.hpp`, `cpp/key_index.cpp`, `experiments/`, `results/scaling-msmarco-1m/`, `results/fiqa-2026-09-09-dense/`, and `calculator/recall-validation.json`. Reproduction and source hashes are described in Section 7.